

Katedra Architektury Systemów Komputerowych

Oprogramowanie systemowe

Materiały pomocnicze do wykładu cz. Ia

Opracował dr inż. Andrzej Jędruch

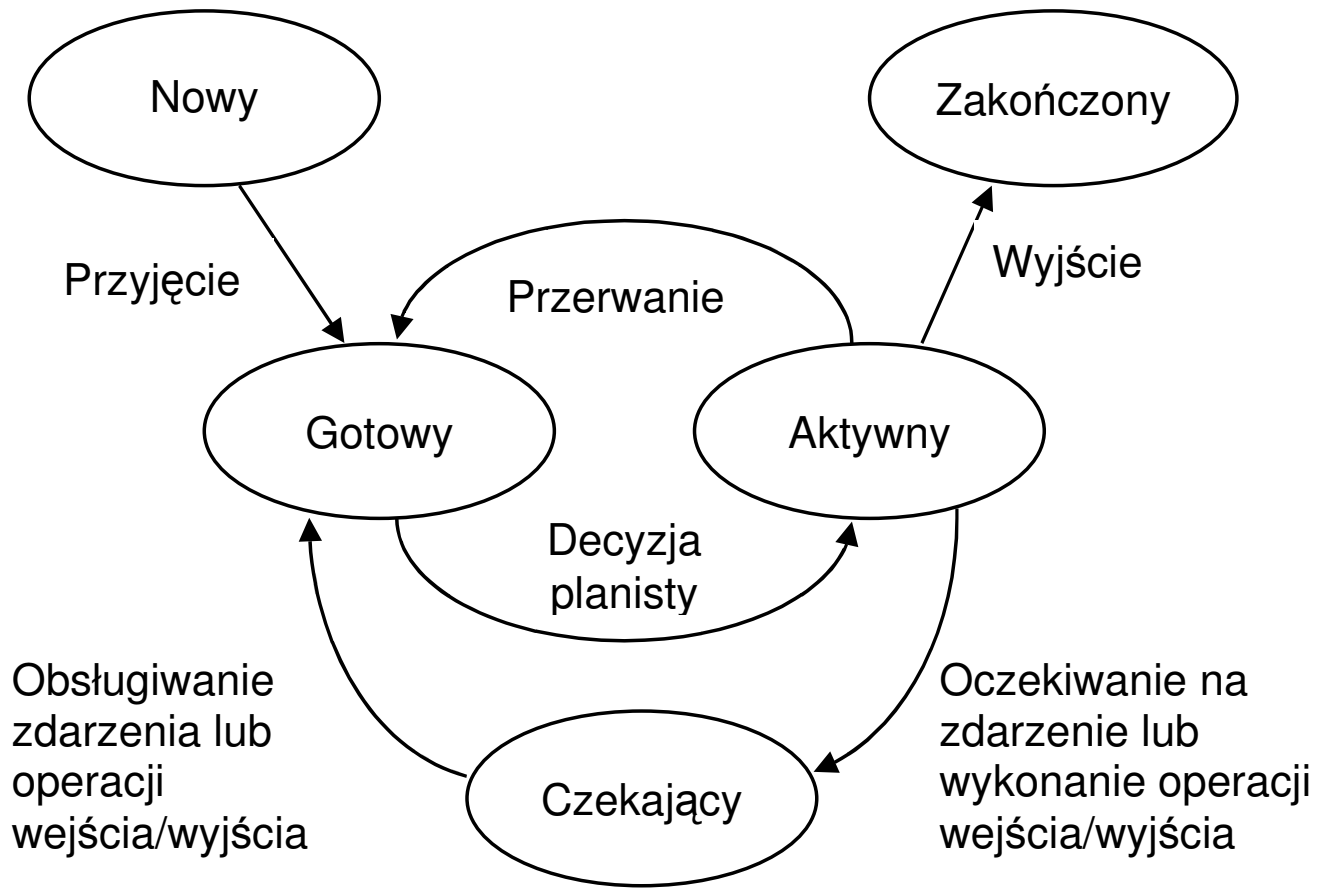
Gdańsk 2007

Procesy

- ◆ współczesne systemy komputerowe pozwalają na umieszczenie w pamięci wielu programów i współbieżne ich wykonywanie; najważniejszym pojęciem, związanym ze wszystkimi systemami operacyjnymi, jest *proces*, którym nazywać będziemy każdy program w stadium wykonywania;
- ◆ sam program jest obiektem pasywnym i nie jest procesem, natomiast proces jest obiektem aktywnym, który ma przydzielone pewne zasoby w systemie i dla którego można wskazać następny rozkaz do wykonania poprzez określenie zawartości wskaźnika instrukcji (licznika rozkazów) procesora;
- ◆ program w formacie wykonywalnym (np. EXE) staje się procesem, jeśli system operacyjny wczyta go do pamięci głównej (operacyjnej) i uruchomi; można więc powiedzieć, że proces składa się z działającego programu oraz bieżących danych o jego stanie i zasobach; wykonywany proces może tworzyć wiele nowych procesów;
- ◆ w niektórych systemach operacyjnych zamiast terminu *proces* używa się terminu *zadanie* — wielozadaniowy system komputerowy może pseudo-równocześnie wykonywać więcej niż jedno zadanie (proces);
- ◆ wielozadaniowość w komputerach osobistych jest w pełni użyteczna tylko w środowisku okien graficznych, gdzie możliwa jest jednoczesna obserwacja działania kilku programów; uruchamianie kilku zadań w systemach sterowanych poleceniami wprowadzanymi z linii zlecenia wymaga skierowania wyników tych zadań do plików (poprzez przekierowanie) — w przeciwnym razie na ekranie mogłyby pojawić przeplatane wyniki kilku realizowanych zadań.

Stany procesu

- ◆ głównym obowiązkiem systemu operacyjnego jest wykonywanie programów użytkowych; system operacyjny musi jednak realizować także różne zadania systemowe (które wygodniej jest pozostawić poza jądrem systemu); można więc przyjąć, że system składa się z zbioru procesów: procesy systemu operacyjnego wykonują kod systemowy, a procesy użytkowe działają wg kodu należącego do użytkowników;
- ◆ odpowiednio, dla obu ww. klas procesów, procesor pracuje w trybie systemu operacyjnego (np. poziom uprzywilejowania nr 0 w architekturze IA-32) albo w trybie użytkownika (poziom 3) — zmiana trybu procesora nazywana jest *przełączaniem kontekstu*;
- ◆ ponieważ istnieje możliwość podziału mocy obliczeniowej procesora (lub procesorów), więc wszystkie te procesy mogą być wykonywane współbieżnie;
- ◆ każdy proces może znajdować się w jednym z następujących stanów:
 - nowy — proces został utworzony,
 - aktywny — są wykonywane instrukcje,
 - oczekiwanie — proces czeka na wystąpienie jakiegoś zdarzenia (np. na zakończenie operacji wejścia-wyjścia),
 - gotowy — proces czeka na przydział procesora,
 - zakończony — proces zakończył działanie.
- ◆ wymienione stany oznaczane są czasami trochę innymi terminami;
- ◆ w każdej chwili w dowolnym procesorze tylko jeden proces może być aktywny, przy czym może być wiele procesów gotowych lub czekających;



♦ system operacyjny kolejno przydziela procesor poszczególnym zadaniom wg przyjętych reguł — ponieważ zmiany następują dość często (kilkanaście razy w ciągu sekundy), użytkownik ma wrażenie, że zadania wykonywane są jednocześnie; znane są dwa podstawowe sposoby przełączania zadań (procesów):

- *wielozadaniowość z wywłaszczaniem* (ang. preemptive multitasking) — w klasycznych wielozadaniowych systemach operacyjnych każdy proces wykonywany jest przez ustalony odcinek czasu (np. 20 ms) — po upływie tego czasu, co jest sterowane przerwaniem zegarowymi, realizowany proces zostaje zatrzymany (zawieszony), po czym wznowiany jest inny proces; niekiedy procesowi może być przydzielony odcinek czasu będący wielokrotnością jednostki podstawowej;

- *wielozadaniowość bez wywłaszczania* (ang. non-preemptive multitasking, cooperative multitasking) — przełączanie realizowane jest za wiedzą programu, za pomocą funkcji systemowej wywołanej przez ten program — zatem proces jest realizowany aż do chwili, gdy dobrowolnie przekaze sterowanie do systemu, który z kolei wznowi inny proces;
- ♦ wielozadaniowość bez wywłaszczania może być stosowana tylko wówczas, jeśli program jest tak skonstruowany, że po wykonaniu krótkiej operacji niezwłocznie przekaze sterowanie do systemu; cechą taką posiadają programy sterowane komunikatami, które są charakterystyczne dla systemu Windows; przyjmuje się, że obsługa pojedynczego komunikatu nie powinna zajmować więcej niż 100 ms; w przypadku błędów może nastąpić "zamrożenie" całego systemu, jeśli program nie przekaze sterowania do systemu;
- ♦ w typowych systemach operacyjnych proces reprezentowany jest przez *blok kontrolny procesu* (ang. process control block – PCB), nazywany również blokiem kontrolnym zadania; w skład bloku kontrolnego wchodzi następujące informacje:

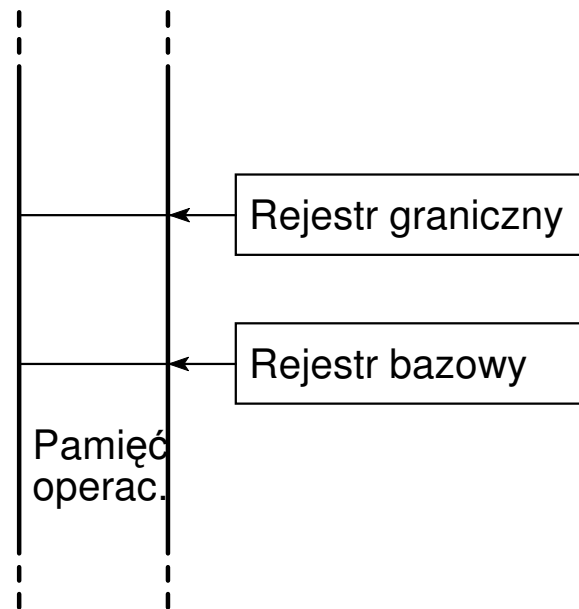
Wskaźnik	Stan procesu
Numer procesu	
Wskaźnik instrukcji	
Rejestry	
Ograniczenia pamięci	
Wykaz otwartych plików	

- stan procesu: może być określony jako nowy, gotowy, aktywny, oczekiwanie, zatrzymanie, itd.;
- wskaźnik instrukcji (licznik rozkazów): wskazuje adres następnego rozkazu do wykonania w procesie;
- rejestry procesora: rodzaje przechowywanych rejestrów zależą od architektury procesora;
- informacja o planowaniu przydziału procesora: priorytet procesu i inne parametry potrzebne do planowania;

- informacje o zarządzaniu pamięcią: zawartości rejestrów granicznych, tablice stron lub segmentów, itp.;
- informacje do rozliczeń: ilość zużytego czasu procesora i czasu rzeczywistego, ograniczenia czasowe, numery kont, itd.;
- informacje o stanie wejścia-wyjścia: informacje o otwartych plikach, stosowanych urządzeniach, itd.

Sprzętowe wspomaganie pracy systemu operacyjnego

- ◆ stosowane są środki sprzętowe i programowe zabezpieczające system operacyjny i realizowane zadania przed zamierzonymi lub przypadkowymi ingerencjami innych zadań; między innymi procesy mogą komunikować się ze sprzętem wyłącznie za pośrednictwem funkcji usługowych systemu operacyjnego; także komunikacja między procesami odbywa się za pośrednictwem systemu operacyjnego;
- ◆ współczesne systemy operacyjne wykonują wiele złożonych operacji, których sprawne wykonanie wymaga pomocy ze strony sprzętu; można tu wymienić poziomy ochrony i związane z nimi specjalne przywileje dla systemu operacyjnego, środki techniczne zabezpieczające wykonywane programy przed wzajemną interferencją, transformacje adresów stanowiące podstawę implementacji pamięci wirtualnej, i wiele innych;
- ◆ system operacyjny nadzoruje pracę sprzętu, i podejmuje odpowiednie działania w przypadku jakichkolwiek prób naruszenia mechanizmów ochrony;
- ◆ jednym z najbardziej znanych, stosowanych od wielu lat sposobów izolacji procesów jest zastosowanie *rejestru bazowego* i *rejestru granicznego*; rejestry ograniczają dostęp procesu do pamięci głównej (operacyjnej) określając dopuszczalny zakres adresów, do których może odwoływać się proces — proces nie powinien mieć dostępu do obszarów pamięci używanych przez system operacyjny i inne procesy;
- ◆ omawianą koncepcję ilustruje poniższy rysunek; rejestr bazowy przechowuje najmniejszy dopuszczalny adres fizyczny pamięci, a rejestr graniczny zawiera największy dopuszczalny adres fizyczny (albo rozmiar obszaru pamięci);



- ◆ koncepcja rejestru bazowego i rejestru granicznego w różnych architekturach procesorów implementowana jest za pomocą rozmaitych środków technicznych; między innymi w architekturze IA-32 koncepcja realizowana jest poprzez określenie adresu bazowego i rozmiaru segmentu w odpowiednich polach (opisanego dalej) deskryptora segmentu.

Segmentacja

- ♦ w trakcie wykonywania pewnego programu przez procesor obserwowano przez krótki odcinek czasu stany linii adresowych (20-bitowych) pamięci głównej i odczytano poniższe wartości:

0010 0011 1011 1011 1101	(23BBD)
0001 1101 0000 0111 0100	(1D074)
0010 0011 1011 1011 1111	(23BBF)
0010 0011 1011 1100 0010	(23BC2)
0010 0011 1011 1011 1101	(23BBD)
0001 1101 0000 0111 1000	(1D078)
0010 0011 1011 1011 1111	(23BBF)
0010 0011 1011 1100 0010	(23BC2)
0010 0011 1011 1011 1101	(23BBD)
0001 1101 0000 0111 1100	(1D07C)

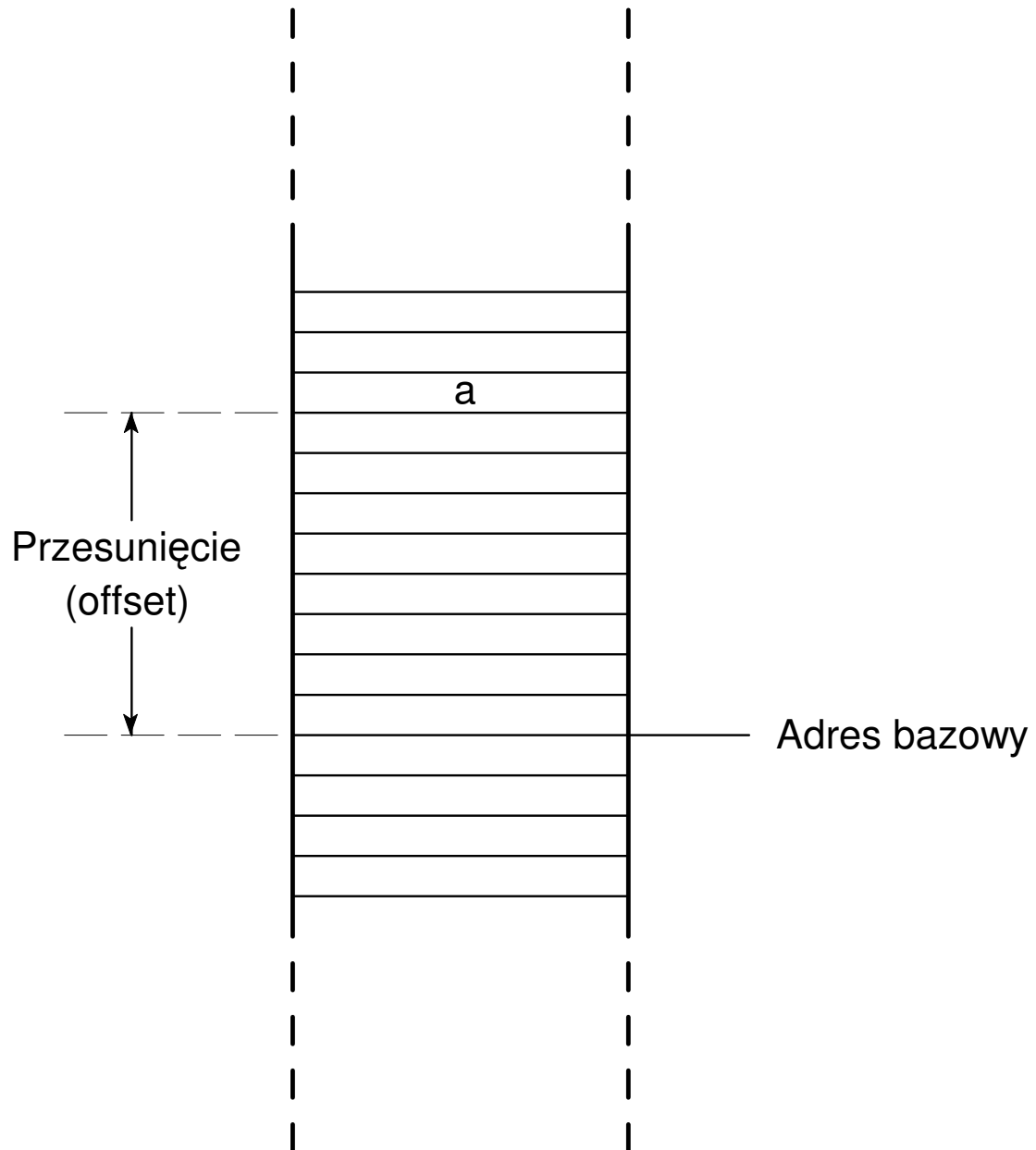
- ♦ łatwo zauważyć, że podane adresy skupiają się w dwóch obszarach:

0010 0011 1011 1011 1101	
	0001 1101 0000 0111 0100
0010 0011 1011 1011 1111	
0010 0011 1011 1100 0010	
0010 0011 1011 1011 1101	
	0001 1101 0000 0111 1000
0010 0011 1011 1011 1111	
0010 0011 1011 1100 0010	
0010 0011 1011 1011 1101	
	0001 1101 0000 0111 1100

- ♦ ponieważ w trakcie wykonywania programu pobierane są kolejne rozkazy programu, umieszczone często w sąsiednich lokacjach pamięci, więc adresy określające położenie rozkazów w pamięci (generowane przez procesor) będą miały zbliżone wartości — można przypuszczać, że lewa kolumna zawiera te właśnie adresy;

- ◆ jednocześnie procesor generuje także adresy danych, które także mają skłonność do skupiania się (zasada lokalności); można więc przypuszczać, że adresy podane w prawej kolumnie określają położenie danych;
- ◆ zauważmy, że zarówno w przypadku adresów rozkazów jak i danych, znaczna część początkowych bitów adresu jest jednakowa w kolejnych odwołaniach do pamięci; zatem istnieje możliwość rozdzielenia obu rodzajów adresów na dwie części: *stałą*, która w ogóle nie zmienia się w trakcie wykonywania programu lub zmienia się bardzo rzadko, i *zmienną*, która ulega zmianom;
- ◆ przy takim podziale wskazanie położenia danej w pamięci czy też położenia rozkazu wymaga podania mniejszej liczby bitów, wskutek czego programy stają się krótsze, zajmują mniej miejsca w pamięci i zwykle wykonują się szybciej; oczywiście procesor we trakcie obliczania adresu lokacji pamięci musi każdorazowo wyznaczyć sumę części stałej i zmiennej;
- ◆ omawiana obserwacja stała się podstawą schematu wyznaczania adresów fizycznych, który został zastosowany w procesorze Intel 8086/88 i później stał się elementem architektury IA-32;
- ◆ w omawianym schemacie części stałe adresów przechowywane są w specjalnych rejestrach procesora, nazywanych *rejestrami segmentowymi*: CS, DS, ES, FS, GS i SS;
- ◆ zawartość rejestru segmentowego DS (ang. data segment) można traktować jako wskazanie pewnego adresu bazowego w pamięci, wskazującego początek obszaru danych, analogicznie zawartość rejestru CS (ang. code segment) wskazuje początek obszaru zawierającego instrukcje (rozkazy) programu;

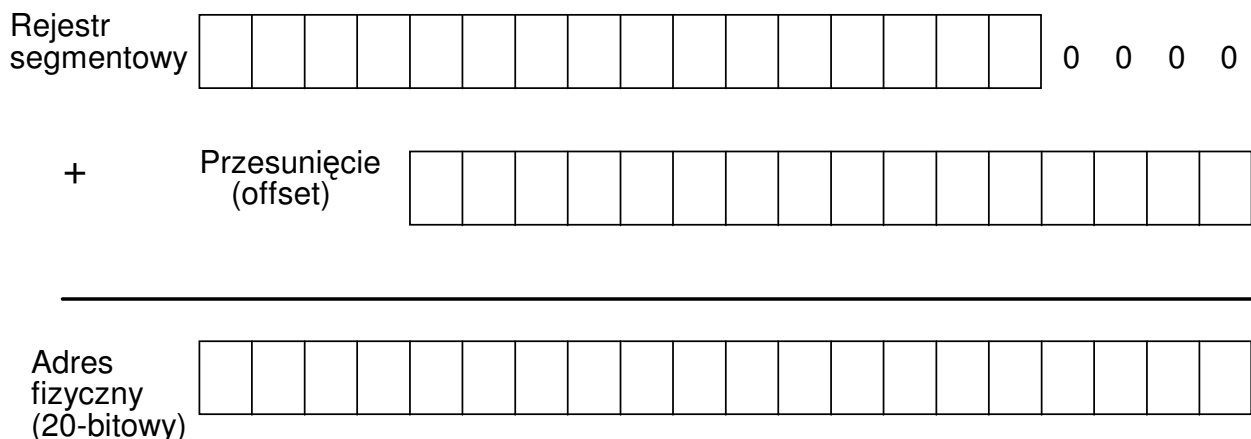
- ◆ w takim ujęciu adresy pamięci używane przez programy określają tylko położenie danej lub rozkazu poprzez podanie odległości, liczonej w bajtach, od adresu bazowego — odległość ta nazywana jest *przesunięciem* lub *offsetem*;
- ◆ zatem adres zawarty w rozkazie (instrukcji) nie zawiera adresu fizycznego danej, czyli nie wskazuje bezpośrednio jej położenia, lecz jedynie odległość danej od pewnego adresu bazowego w pamięci komputera;
- ◆ przykładowo, jeśli instrukcja (rozkaz) wykonująca pewne działanie na zmiennej *a* zawiera adres tej zmiennej, to adres tej zmiennej wyrażony poprzez odległość tej zmiennej, liczoną w bajtach, od pewnego ustalonego adresu bazowego; zatem wyznaczenie adresu fizycznego zmiennej *a* wymaga obliczenia sumy adresu bazowego i przesunięcia; w procesorach zgodnych z architekturą IA-32 suma ta wyznaczana jest w specyficzny sposób.



Wyznaczanie adresów fizycznych w trybie rzeczywistym

- ◆ adres przekazywany do podzespołów pamięci w celu wskazania położenia bajtu, który ma zostać odczytany lub zapisany nosi nazwę *adresu fizycznego*, a zbiór wszystkich adresów fizycznych nazywa się *fizyczną przestrzenią adresową*;
- ◆ w architekturze IA-32 przyjęto, że adresy fizyczne są liczbami 32-bitowymi i mogą przyjmować wartości z przedziału $<0, 4\,294\,967\,295>$; w takim przypadku mówimy, że rozmiar fizycznej przestrzeni adresowej wynosi 4 GB;
- ◆ w nowszych typach procesorów omawianej architektury liczbę linii adresowych zwiększono do 36 i więcej, co powoduje odpowiednie zwiększenie fizycznej przestrzeni adresowej; w dalszych rozważaniach ograniczymy się jednak do konwencjonalnej 4 GB przestrzeni adresowej, a adresowanie w większych obszarach omówimy jako przypadek specjalny;
- ◆ architektura IA-32 rozwinęła się z opracowanej w r. 1978 architektury procesorów 8086 i 8088, w których zdefiniowano przestrzeń adresową 1 MB (20 linii adresowych); ze względu na kompatybilność wsteczną współczesne procesory, zgodne z architekturą IA-32, nadal mogą pracować w sposób prawie identyczny jak ww. procesory;
- ◆ omawiany tryb pracy, określany jako *tryb rzeczywisty*, uruchamiany jest bezpośrednio po włączeniu (lub zresetowaniu) komputera — w trakcie uruchamiania systemu operacyjnego (np. Windows lub Linux) procesor zostaje przełączony z trybu rzeczywistego do trybu chronionego, który pozwala wykorzystać w pełni możliwości architektury IA-32;

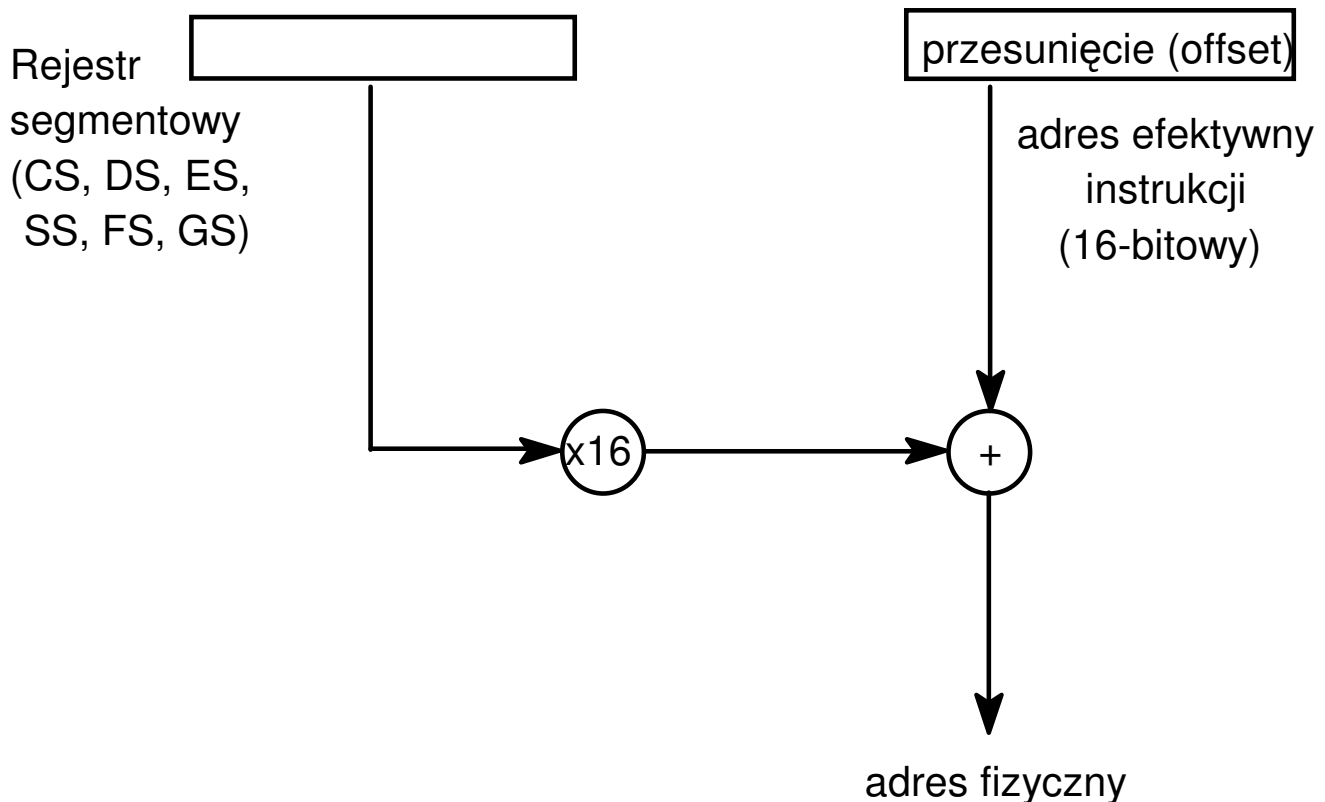
- ◆ jak stwierdziliśmy wcześniej, wyznaczenie adresu fizycznego pewnej zmiennej **a** wymaga obliczenia sumy adresu bazowego i przesunięcia; taki sposób wyznaczania adresu fizycznego jest charakterystyczny dla trybu rzeczywistego, natomiast w trybie chronionym wyznaczanie adresu fizycznego stanowi złożony proces, który będzie szczegółowo omawiany dalej;
- ◆ ponieważ w trybie rzeczywistym adresy fizyczne są 20-bitowe, należałoby więc przypuszczać, że omawiane wcześniej rejestry segmentowe CS, DS, ES, FS, GS i SS, które wskazują adresy bazowe segmentów są rejestrami 20-bitowymi; konstruktorzy procesora ograniczyli jednak długość tych rejestrów do 16 bitów, przyjmując jednocześnie założenie, że w obliczeniach adresów zawartość każdego z ww. rejestrów segmentowych jest rozszerzana do 20 bitów poprzez dopisanie czterech zer z prawej strony;
- ◆ przyjęto także, że w trybie rzeczywistym przesunięcie (offset) jest liczbą 16-bitową;
- ◆ przy podanych założeniach sposób obliczania adresu fizycznego ilustruje poniższy rysunek:



- ♦ zauważmy, że dopisanie czterech zer z prawej strony liczby dwójkowej oznacza po prostu mnożenie przez 16; zatem adres fizyczny w trybie rzeczywistym określa formuła:

$\text{adres fizyczny} = \text{zaw. rej. segmentowego} * 16 + \text{przesunięcie}$

- ♦ podaną zależność ilustruje poniższy rysunek:



- ♦ w praktyce programowania (w trybie rzeczywistym) zazwyczaj nie podaje się 20-bitowego adresu fizycznego instrukcji lub danej, ale adres ten wyraża się w postaci dwóch liczb szesnastkowych rozdzielonych dwukropkiem; pierwsza z tych liczb określa wskazuje początek segmentu, w którym znajduje omawiany obiekt, druga zaś określa przesunięcie wewnątrz segmentu, tj. odległość obiektu od początku segmentu (wyrażona w bajtach);
- ♦ przykładowo, 32-bitowy programowy licznik czasu BIOSu umieszczony jest w lokacji pamięci o adresie 40H:6CH, tzn. w lokacji pamięci o adresie fizycznym $40H \times 16 + 6CH = 46CH$; mówimy, że adres lokacji pamięci został wyrażony w postaci *segment : offset*.

Wykonywanie programów w środowisku wielozadaniowym

- ◆ przyjmijmy, że pamięć główna (operacyjna) komputera jest wystarczająco duża by pomieścić w niej kilka aktualnie wykonywanych programów (procesów);
- ◆ w trakcie kompilacji i konsolidacji tych programów nie jest znane ich położenie w pamięci w trakcie wykonywania w systemie wielozadaniowym; w tej sytuacji konsolidatory zakładają, że program zostanie umieszczony w pamięci głównej począwszy od adresu 0;
- ◆ decyzję o załadowaniu kodu i danych programu do wybranego obszaru pamięci głównej podejmuje system operacyjny; znane są różne algorytmy wyboru najbardziej odpowiedniego obszaru, jednak z reguły obszar ten nie zaczyna się od adresu 0;
- ◆ w tej sytuacji wydaje się konieczne wykonanie ponownej translacji programu, tak by adresy kodu i danych były dostosowane do położenia przydzielonego obszaru pamięci;
- ◆ ponowna translacja jest możliwa tylko w wyjątkowych przypadkach, gdy użytkownik komputera posiada kod źródłowy programu; z reguły jednak kod źródłowy jest niedostępny;
- ◆ pozostaje więc przyjęcie założenia, że program będzie umieszczony w pamięci począwszy od ustalonego adresu, np. od adresu 0;
- ◆ ponieważ inne programy przygotowywane są w ten sam sposób, z pewnością pojawią się konflikty w dostępie do pamięci; znane są trzy typowe rozwiązania tego problemu:

- translator dostarcza kod programu w postaci *relokowalnej*, tzn. przygotowanej do umieszczenia w dowolnym obszarze pamięci — zazwyczaj system operacyjny ładuje program do niezajętego obszaru pamięci, odpowiednio zmieniając przy tym adresy niektórych rozkazów;
 - w przypadku mniejszych programów, w niektórych architekturach procesorów (m.in. IA-32) możliwe jest wytworzenie kodu programu, który może być załadowany do dowolnego obszaru pamięci bez relokacji (ang. position independent code);
 - dostarczony kod programu nie jest relokowalny, a odwzorowanie kodu i danych poszczególnych programów w odrębne obszary pamięci fizycznej wykonywane jest przez procesor w trakcie wykonywania programu (z pewną pomocą systemu operacyjnego).
- ◆ używanie kodu relokowalnego może być kłopotliwe, jeśli konieczne jest przeniesienie programu do innego obszaru w trakcie jego wykonywania; z tego powodu podstawowe znaczenie mają metody odwzorowywania adresów wykonywane przez procesor;
- ◆ w tej sytuacji za pomocą mechanizmów adresowania procesora trzeba stworzyć iluzję dla programu, że znajduje się on w obszarze pamięci o adresie początkowym zero, podczas w gdy rzeczywistości obszar ten może znajdować daleko od adresu 0;
- ◆ musi więc nastąpić wyodrębnienie dwóch kategorii adresów komórek pamięci: adresów fizycznych używanych przez podzespoły komputera do wskazania pewnej komórki w pamięci operacyjnej i adresów wirtualnych, którymi posługują się programiści.

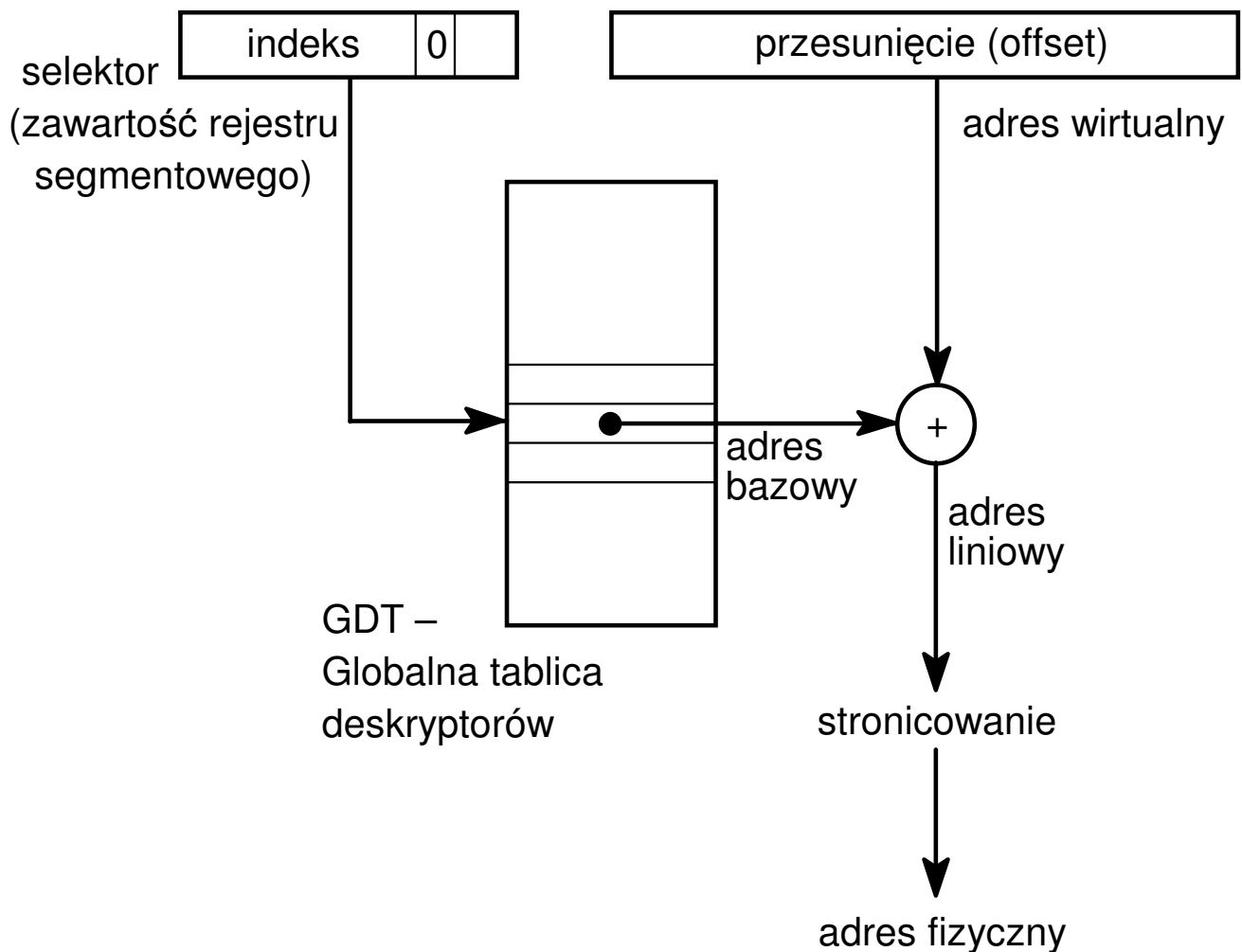
Adresy fizyczne i wirtualne

- ◆ mechanizmy adresowania pamięci stosowane we współczesnych procesorach są bardzo złożone; programista tworzący aplikacje w wielu przypadkach nie zna szczegółów związanych z wyznaczaniem adresów przez procesor i posługuje się uproszczonym modelem pamięci, w którym cała pamięć jest adresowana bezpośrednio poprzez wskazanie adresu potrzebnego bajtu w pamięci głównej (operacyjnej);
- ◆ adresy, którymi posługuje się programista w trakcie kodowania rozkazów kodu maszynowego noszą nazwę *adresów wirtualnych*; takie adresy występują również w kodzie programu wytworzonym przez kompilator języka wysokiego poziomu;
- ◆ zbiór wszystkich adresów wirtualnych (logicznych) generowanych przez program nazywany jest *wirtualną przestrzenią adresową*; z reguły wirtualna przestrzeń adresowa jest znacznie większa niż rozmiar zainstalowanej pamięci głównej (RAM);
- ◆ w prostych systemach adresy wirtualne i fizyczne są utożsamiane; jednak w systemach wielozadaniowych konieczne jest rozdzielenie adresów wirtualnych i fizycznych, co wymaga wprowadzenia mechanizmów odwzorowywania adresów wirtualnych na fizyczne; operacje te wykonywane są przez moduł zarządzania pamięcią MMU (ang. memory management unit) w procesorze.

Obliczanie adresu fizycznego w trybie chronionym

- ◆ wyznaczanie adresu fizycznego w trybie chronionym jest znacznie bardziej skomplikowane niż w trybie rzeczywistym; obejmuje mechanizmy stronicowania i segmentacji: adres wskazany przez instrukcję programu zostaje przekształcony poprzez specjalne tablice segmentów na *adres liniowy*, a ten z kolei poprzez mechanizmy stronicowania na *adres fizyczny*;
- ◆ z punktu widzenia systemu operacyjnego mechanizmy segmentacji i stronicowania mają podobne zastosowania; jednak współczesne systemy operacyjne preferują mechanizmy stronicowania, które pozwalają na bardziej efektywne zarządzanie pamięcią — w takiej sytuacji mechanizmy segmentacji zostają ograniczone do minimum, a tworzone segmenty często pokrywają się z całą dostępną przestrzenią adresową;
- ◆ w pierwszej kolejności rozpatrzmy mechanizmy segmentacji, jednocześnie zakładając, dla uproszczenia rozważań, że stronicowanie jest nieaktywne; warto dodać, że stronicowanie może zostać opcjonalnie wyłączone (bit 31 w rejestrze CR0), natomiast nie ma możliwości wyłączenia segmentacji;
- ◆ w architekturze IA-32, z punktu widzenia programu, położenie lokacji pamięci określone jest przez dwa elementy tworzące tzw. *adres logiczny*:
 - wskazanie segmentu zawierającego tę lokację,
 - wskazanie przesunięcia (offsetu) lokacji od początku segmentu;
- ◆ konkretny segment wskazywany jest przez 16-bitowy selektor zawarty w jednym z rejestrów segmentowych; jednak położenie tego segmentu w pamięci, inaczej niż w trybie rzeczywistym, wyznaczane jest za pomocą specjalnej tablicy;

- ◆ przesunięcie względem początku segmentu określa 16- lub 32-bitowy *adres wirtualny* (nazywany też adresem efektywnym lub offsetem), który powstaje w wyniku sumowania adresu zawartego w instrukcji i zawartości rejestrów modyfikacji;
- ◆ w wyniku sumowania adresu początkowego (bazowego) segmentu i adresu wirtualnego uzyskuje się *adres liniowy*, który przy wyłączonym stronicowaniu jest jednocześnie *adresem fizycznym*.



Globalna tablica deskryptorów (GDT)

- ◆ podobnie jak w trybie rzeczywistym, każdy program działa na segmentach — rejestry segmentowe nie wskazują jednak bezpośrednio konkretnej lokacji pamięci, lecz element pewnej tablicy zwany *deskryptorem segmentu*;
- ◆ każdy deskryptor segmentu zawiera 8 bajtów, wśród których 4 bajty określają 32-bitowy adres bazowy segmentu;

bity 31 ÷ 24 adresu bazowego		+6
	bity 23 ÷ 16 adresu bazowego	+4
bity 15 ÷ 0 adresu bazowego		+2
		+0

- ◆ struktura deskryptora została pierwotnie zaprojektowana dla procesora 80286, w którym określa się 24-bitowy adres bazowy (16 MB); ponieważ przy rozszerzaniu adresu bazowego do 32 bitów należało zachować istniejącą strukturę, nowa struktura stała się niesystematyczna, co komplikuje operacje związane z deskryptorem;
- ◆ deskryptory segmentów są zgrupowane w *Globalnej Tablicy Deskryptorów* (GDT), a w bardziej skomplikowanych systemach występują także w *Lokalnych Tablicach Deskryptorów* (LDT);

- ♦ zawartość rejestru segmentowego w trybie chronionym, wskazuje pewien deskryptor w globalnej (lub lokalnej) tablicy deskryptorów; zawartość rejestru segmentowego w trybie chronionym nazywana jest *selektorem*:

indeks	TI	RPL
13 bitów		2 bity

- ♦ pole *indeks* w selektorze wskazuje deskryptor w globalnej (gdy bit TI = 0) lub w lokalnej (gdy bit TI = 1) tablicy deskryptorów; pole RPL (ang. Requester Privilege Level) stanowi jeden z elementów mechanizmu ochrony;
- ♦ adres bazowy globalnej tablicy deskryptorów GDT wskazuje rejestr GDTR:

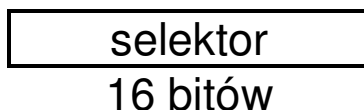
adres liniowy wskazujący położenie tablicy GDT	(wielkość GDT) – 1
32 bity	16 bitów

- ♦ pole *indeks* umożliwia wskazanie jednego z 8192 deskryptorów — zatem tablica GDT może zajmować maksymalnie 65536 bajtów — ponieważ zapis binarny tej liczby zawiera 17 bitów, stąd pole *wielkość* w rejestrze GDTR podaje rozmiar tablicy GDT pomniejszony o 1, co pozwala na zapisanie rozmiaru na 16 bitach;
- ♦ jeśli GDT zawiera k deskryptorów, to są one numerowane 0, 1, 2, ..., $k - 1$; często w literaturze technicznej zamiast numeru deskryptora podaje się selektor wskazujący na dany deskryptor np.: ... 40H, 48H, 50H, 58H, 60H, ...; ponieważ dwa najmłodsze bity selektora nie określają deskryptora, więc selektory 50H, 51H, 52H i 53H wskazują ten sam deskryptor; analogicznie inne deskryptory;

- ◆ *selektor zerowy*: selektor w którym pole *indeks* i pole TI zawierają zera; selektor zerowy nie wskazuje deskryptora zerowego; załadowanie selektora zerowego do rejestru segmentowego jest poprawne, ale próba użycia tego rejestru do adresowania spowoduje wygenerowanie wyjątku (przerwania); zwykle na początku zadania selektory zerowe wpisywane są do rejestrów DS, ES, FS, GS;
- ◆ gdy pole TI selektora zawiera 0, to selektor odnosi się do globalnej tablicy deskryptorów GDT; wtedy na podstawie zawartości wskazanego deskryptora w GDT wyznacza się 32-bitowy adres bazowy segmentu; poprzez sumowanie uzyskanego adresu bazowego i adresu wirtualnego (16- lub 32-bitowego) uzyskuje się adres liniowy, który przy wyłączonym stronicowaniu można interpretować jako adres fizyczny.

Lokalne tablice deskryptorów

- ♦ w dużych systemach oprogramowania obok globalnej tablicy deskryptorów używane są także lokalne tablice deskryptorów LDT (ang. local descriptor table);
- ♦ gdy pole TI selektora zawiera 1, wtedy adres bazowy potrzebnego segmentu zawarty jest w deskrypcie umieszczonym w jednej z lokalnych tablic deskryptorów, spośród których tylko jedna może być aktywna; aktywna lokalna tablica deskryptorów wskazywana jest przez 16-bitowy rejestr LDTR:



- ♦ rejestr LDTR zawiera selektor wskazujący deskryptor w tablicy GDT; z kolei deskryptor ten wskazuje segment zawierający lokalną tablicę deskryptorów LDT – w tej właśnie tablicy odszukiwany jest deskryptor wskazany przez pole *indeks* selektora umieszczonego w rejestrze segmentowym; następnie zawarty tam adres bazowy segmentu jest sumowany z adresem wirtualnym i dalsze działania są identyczne jak w przypadku TI = 0;
- ♦ rejestr LDTR ładowany jest za pomocą instrukcji `lldt` (ang. load local descriptor table register);
- ♦ segmenty LDT nie mogą być bezpośrednio odczytywane, zapisywane lub wykonywane — mogą być używane tylko w procesie translacji adresu; jednocześnie, do rejestru LDTR mogą być ładowane tylko selektory wskazujące segmenty LDT; zatem by utworzyć lokalną tablicę deskryptorów, system operacyjny musi najpierw utworzyć w GDT deskryptor segmentu danych i wpisać deskryptory do tego segmentu; następnie system operacyjny musi zmienić (omawiany dalej) bajt dostępu w taki sposób, by utworzony segment stał się segmentem LDT.

Rozmiar segmentu i tryb adresowania

- ♦ rozmiar segmentu przechowywany jest w deskrytorze segmentu w postaci zakodowanej w formie liczby 20-bitowej, która umieszczona jest w polu *wielkość segmentu*;

	G D 0 V	wlk.segm. 19 – 16	+6
			+4
			+2
			+0
wielkość segmentu (bity 15 – 0)			

- ♦ rozmiar segmentu określa w istocie górną granicę segmentu — jakiegokolwiek działanie dotyczące lokacji pamięci poza granicami segmentu powodują wygenerowanie wyjątku; w segmentach rozszerzalnych w dół (które omawiane są dalej) stosowane są odmienne zasady;
- ♦ w architekturze IA-32 przyjmuje się, że rozmiar segmentu jest zawsze liczbą 32-bitową — bit G określa sposób transformacji zawartości 20-bitowego pola *wielkość segmentu* na 32-bitowy rozmiar segmentu:
 - G = 0 12 najstarszych bitów zawiera 0, natomiast pozostałe 20 bitów określone jest przez pole *wielkość segmentu* (jest to tzw. ziarnistość bajtowa, w której wielkość segmentu nie przekracza 1MB);
 - G = 1 12 najmłodszych bitów zawiera 1, natomiast pozostałe 20 określone są przez pole *wielkość segmentu* (w tym przypadku wielkość segmentu określona jest w deskrytorze za pomocą jednostek 4 KB).

- ♦ uzyskana w ten sposób liczba 32-bitowa określa rozmiar segmentu pomniejszony o 1; zatem, jeśli pole *wielkość segmentu* zawiera liczbę 0, to rzeczywista wielkość segmentu wynosi 1 bajt (gdy $G = 0$) albo 4096 bajtów (gdy $G = 1$);
- ♦ z bitem G sąsiaduje bit D (czasami oznaczany literą B), który określa standardowy rozmiar operandu i adresu dla wszystkich rozkazów zawartych w segmencie:

	G	D	0	V	wlk.segm.	+6
	7	6	5	4	19 – 16	

$D = 0$ adresy 16-bitowe, operandy 8- i 16-bitowe

$D = 1$ adresy 32-bitowe, operandy 8- i 32-bitowe

- ♦ rozmiary adresów i operandów mogą być doraźnie zmienione poprzez stosowanie odpowiednich przedrostków — zagadnienia te omawiane były na wykładzie z „Architektury komputerów”;
- ♦ zalecane jest ażeby w przypadku segmentu rozszerzalnego w dół bity G i D były jednakowe, czyli
 $ED = 1, G = 0, D = 0$, albo
 $ED = 1, G = 1, D = 1$;
- ♦ bit V jest niezdefiniowany i może być używany przez oprogramowanie.

Rejestry ukryte

- ◆ algorytm wyznaczania adresu fizycznego w trybie chronionym jest znacznie bardziej skomplikowany od algorytmu dla trybu rzeczywistego – można więc przypuszczać, że czas wykonywania instrukcji będzie dłuższy; wykorzystanie rejestrów ukrytych znacznie usprawnia mechanizm adresowania;
- ◆ w typowych programach odwołania do pamięci występują znacznie częściej niż zmiany rejestrów segmentowych;
- ◆ gdy program ładuje selektor do rejestru segmentowego, następuje sprawdzenie czy deskryptor wskazywany przez ten selektor jest odpowiedni dla wykonywanego programu — m.in. porównywany jest poziom uprzywilejowania (zob. dalszy opis) segmentu i programu, typ deskryptora (niektóre deskryptory nie opisują segmentów) a także sprawdzana jest obecność segmentu w pamięci; stwierdzenie jakiegokolwiek niezgodności powoduje wygenerowanie wyjątku (przerwania), zazwyczaj typu GP (ang. general protection);
- ◆ jeśli sprawdzenie zakończy się pomyślnie, to analizowany deskryptor zapisywany jest rejestrze pomocniczym w procesorze; dla każdego rejestru segmentowego, a także dla rejestru LDTR istnieje odrębny rejestr pomocniczy; w trakcie realizacji dalszych instrukcji programu procesor korzysta z adresów, wielkości segmentów i atrybutów zapisanych w rejestrach pomocniczych, co znacznie skraca czas wykonywania instrukcji; rejestry pomocnicze nie są dostępne dla programisty, stąd nazywane są zazwyczaj *rejestrami ukrytymi* (ang. shadow registers).

Koncepcja pamięci wirtualnej

- ◆ praca w trybie wielozadaniowym oznacza pseudo-jednoczesne wykonywanie przez procesor kilku zadań;
- ◆ znaczna część programów wykonywanych przez komputery wykazuje zmienne zapotrzebowanie na pamięć główną (operacyjną) — programy kierują do systemu operacyjnego żądania przydzielenia nowych obszarów pamięci, które po pewnym czasie, jeśli są już niepotrzebne, zwracane są do systemu; niekiedy programy żądają przydzielenia obszarów większych niż cała zainstalowana pamięć główna (operacyjna);
- ◆ wykonywanie programów w trybie wielozadaniowym wymaga udostępnienia każdemu programowi wymaganego obszaru pamięci głównej (operacyjnej);
- ◆ pamięć główna stanowi jednak dość ograniczony zasób systemu, koszt jej zainstalowania jest wielokrotnie większy niż innych pamięci, i stąd system operacyjny powinien racjonalnie i efektywnie zarządzać tą pamięcią;
- ◆ wymienione czynniki powodują, że konieczne staje się symulowanie dużej pamięci głównej za pomocą pamięci RAM i pamięci dyskowej — utworzona w ten sposób *pamięć wirtualna* widziana jest przez programy jako zwykła pamięć operacyjna;
- ◆ w architekturze IA-32 zdefiniowano rozbudowane mechanizmy wspomagające tworzenie pamięci wirtualnej — może ona być implementowana za pomocą segmentacji lub stronicowania; pamięć wirtualna oparta na segmentacji stosowana była w starszych wersjach systemu Windows, obecnie stosuje się przede wszystkim bardziej efektywne stronicowanie.

Realizacja pamięci wirtualnej za pomocą segmentacji

- ♦ jedna z technik realizacji pamięci wirtualnej polega na przechowywaniu niektórych segmentów programu w pamięci dyskowej i sprowadzaniu ich do pamięci operacyjnej gdy stają się potrzebne;
- ♦ istotną rolę w takiej realizacji odgrywają bity P, A i W zawarte w deskrypcie segmentu, a dokładniej, w opisanym dalej szczegółowo bajcie dostępu;

P						W	A
7	6	5	4	3	2	1	0

- ♦ obecność segmentu w pamięci operacyjnej wskazuje bit P (ang. present); jeśli w programie wystąpi odwołanie do deskryptora, w którym bit P zawiera 0, to pojawia się wyjątek (przerwanie generowane przez procesor), obsługiwany przez system operacyjny (poprzez deskryptor przerwania 0BH); w takiej sytuacji system operacyjny odszukuje odpowiednio duży niezajęty obszar pamięci operacyjnej, ewentualnie przesyłając na dysk segmenty aktualnie nieużywane, a następnie kopiuje potrzebny segment z dysku do pamięci operacyjnej; po załadowaniu segmentu sterowanie powraca do realizowanego programu; operacje te są całkowicie "niewidoczne" dla wykonywanego programu;
- ♦ racjonalne zorganizowanie wymiany segmentów wymaga posiadania informacji statystycznych o wykorzystaniu poszczególnych segmentów; do zbierania tych informacji może być przydatny bit A (ang. access);

- ◆ bit A ustawiany jest na 1, gdy do rejestru segmentowego zostanie wpisany selektor wskazujący rozpatrywany deskryptor; bit A nie jest nigdy automatycznie zerowany przez procesor; system operacyjny zeruje bity A w obserwowanych segmentach i następnie okresowo odczytuje ich zawartość, każdorazowo wykonując też zerowanie; jeśli dla pewnego segmentu, kilkakrotnie odczytano zerową wartość bitu A, to segment ten jest zapewne nieużywany;
- ◆ w implementacji pamięci wirtualnej może być także pomocny bit W; system operacyjny może inicjalnie przypisać wszystkim segmentom danych atrybut "tylko do odczytu" ($W = 0$) – zatem po uruchomieniu programu jakakolwiek próba zapisu do segmentu danych spowoduje wygenerowanie wyjątku; program obsługi tego wyjątku sprawdzi legalność operacji zapisu w odniesieniu do rozpatrywanego segmentu, i w przypadku pozytywnym ustawi bit $W = 1$, po czym przekaże sterowanie ponownie do wykonywanego programu (od tej chwili operacje zapisu i odczytu w tym segmencie będą wykonywane konwencjonalnie, bez generowania wyjątku);
- ◆ w tej sytuacji stan bitu $W = 1$ stanowi informację dla systemu operacyjnego, że zawartość segmentu została zmieniona; w przypadku, gdy segment będzie musiał być usunięty z pamięci operacyjnej, konieczne jest przepisanie go na dysk; jeśli jednak bit $W = 0$, to oznacza, że reprezentacje segmentu w pamięci operacyjnej i na dysku są jednakowe, wobec czego kopiowanie usuwanego segmentu nie jest potrzebne;
- ◆ implementacja pamięci wirtualnej za pomocą segmentacji pozwala, w środowisku wielozadaniowym, na tworzenie iluzji dla uruchomionych programów, że każdy z nich ma do dyspozycji prawie całą zainstalowaną pamięć operacyjną; jednak wymiana segmentów, ze względu na ich niejednakowe rozmiary może być kłopotliwa; ubocznym skutkiem wymiany segmentów może być fragmentacja pamięci, tj. powstanie dużej liczby niewielkich obszarów pamięci, które nie mają zastosowania.

Bajt dostępu

- ♦ tablice deskryptorów (GDT lub LDT) mogą zawierać deskryptory opisujące obiekty różnych typów, w szczególności mogą to być omawiane wcześniej deskryptory segmentów – wszystkie typy deskryptorów zawierają na tej samej pozycji bajt dostępu, którego kod identyfikuje typ deskryptora:

bity 31 – 24 adresu bazowego	G D 0 V	wlk.segm. 19 – 16	+6
bajt dostępu	bity 23 – 16 adresu bazowego		+4
bity 15 – 0 adresu bazowego			+2
wielkość segmentu (bity 15 – 0)			+0

- ♦ poniżej podano struktury bajtu dostępu dla różnych typów deskryptorów segmentów; znaczenie poszczególnych bitów zostanie omówione w dalszej części.

7	6 - 5	4	3	2	1	0	
P	DPL	1	1	C	R	A	a) bajt dostępu dla segmentu kodu

P	DPL	1	0	ED	W	A	b) bajt dostępu dla segmentu danych
---	-----	---	---	----	---	---	-------------------------------------

P	DPL	0	0	0	1	0	c) bajt dostępu dla segmentu LDT
---	-----	---	---	---	---	---	----------------------------------

Interpretacja bitów w bajcie dostępu

- ◆ dwubitowe pole DPL (ang. descriptor privilege level) jest związane z uprzywilejowaniem segmentów i mechanizmami ochrony — pole to omawiane jest w dalszej części wykładu;
- ◆ w trybie chronionym wyodrębnia się segmenty kodu i segmenty danych, przypisując im odrębne własności;
- ◆ interpretacja bitu R w bajcie dostępu dla segmentu kodu:
 - R = 0 segment kodu może być tylko wykonywany;
 - R = 1 segment kodu może być wykonywany i odczytywany,

w obu przypadkach próba zapisu do segmentu kodu, a także odczytu gdy R = 0, powoduje wygenerowanie wyjątku (przerwania);

- ◆ interpretacja bitu W w bajcie dostępu dla segmentu danych:
 - W = 0 segment danych może być tylko odczytywany,
 - W = 1 segment danych może być odczytywany i zapisywany;
- ◆ w przypadku segmentów o najwyższym poziomie uprzywilejowania (DPL = 0) spotykane zazwyczaj bajty dostępu mają wartość: 9BH dla segmentu kodu i 93H dla segmentu danych;
- ◆ segment kodu może być tylko wykonywany i ewentualnie odczytywany — nie może być natomiast zapisywany (próba takiego zapisu powoduje wyjątek GP – ang. general protection, wektor 0DH); jeśli zachodzi konieczność dokonania zmian w segmencie kodu, to trzeba tymczasowo zmienić bajt dostępu segmentu (np. 9BH zamienić na 93H) albo też zdefiniować odrębny deskryptor dla segmentu danych, który będzie zajmował ten sam obszar co segment kodu; druga metoda znana jest w literaturze jako *alias segmentu kodu*.

Zwiększenie rozmiarów segmentu danych i segmentu stosu

- ◆ jeśli obszar pamięci powyżej segmentu danych statycznych nie jest zajęty, to wystarczy tylko odpowiednio zwiększyć pole *wielkość segmentu*, natomiast jeśli obszar ten jest zajęty, to konieczne jest usunięcie segmentu zajmującego potrzebny obszar albo inne uporządkowanie segmentów w pamięci;
- ◆ w przypadku segmentu stosu zwiększenie jego rozmiaru byłoby znacznie bardziej kłopotliwe — stos bowiem "rośnie w dół", a wartość 0 wskaźnika ESP oznacza całkowite zapełnienie stosu; zwiększenie rozmiarów stosu wymagałoby więc przesunięcia adresu bazowego i odpowiedniej korekcji wskaźnika stosu ESP; dalsze komplikacje mogłyby wynikać ze zmiany położenia wcześniej zapisanych danych, które mogą być odczytywane i zapisywane za pomocą innych rozkazów niż POP i PUSH ;
- ◆ wymienione przyczyny spowodowały wyodrębnienie dwóch typów segmentów danych:
 - segmentów rozszerzalnych w górę (ED = 0, ang. expand up),
 - segmentów rozszerzalnych w dół (ED = 1, ang. expand down).
- ◆ segmenty rozszerzalne w górę przeznaczone głównie do przechowywania danych statycznych — w odniesieniu do tego typu segmentów obowiązują wcześniej omówione zasady interpretacji pola *wielkość segmentu*;
- ◆ segmenty rozszerzalne w dół przeznaczone do przechowywania stosu — w segmentach tego typu pola *adres bazowy* i *wielkość segmentu* opisują, odwrotnie niż dla pozostałych typów, obszar **nie zajmowany** przez segment; ściśle rzecz biorąc, jeśli bit ED = 1, to dopuszczalne są tylko takie odwołania do segmentu, w których offset jest większy od podanego rozmiaru segmentu; koncepcję segmentów rozszerzalnych w dół wyjaśnia poniższy przykład.

Przykład: przypuśćmy, że w pewnym programie wykonywanym na komputerze, w którym zainstalowano 8 MB pamięci RAM zamierzamy zdefiniować segment stosu zawierający 28 KB (tj. 7000H bajtów). Segment ten powinien zajmować obszar pamięci 00010000H ÷ 00016FFFH (tj. począwszy od adresu fizycznego 65536).

FFFFFFFFFH
pozostała część przestrzeni adresowej procesora (4GB)
00800000h
007FFFFFFH
pamięć fizyczna
00017000H
00016FFFH
segment stosu (rozmiar 7000H)
00010000H
0000FFFFH
pamięć fizyczna
00000000H

- ♦ utworzenie segmentu stosu wymaga zdefiniowania odpowiedniego deskryptora; przyjmiemy, że bit G = 1, czyli rozmiar segmentu jest wielokrotnością 4 KB;

- ♦ jako *adres bazowy* należy podać adres pierwszego bajtu **nie należącego** do stosu, czyli 00017000H;
- ♦ obliczenie zawartości pola *wielkość segmentu* jest bardziej skomplikowane; najpierw należy obliczyć rozmiar przestrzeni adresowej **nie zajmowanej** przez segment, czyli

$$100000000H - 7000H = FFFF9000H$$

ponieważ pole *wielkość segmentu* zawiera rozmiar segmentu pomniejszony o 1, więc uzyskaną wartość należy zmniejszyć o 1, przez co uzyskujemy FFFF8FFFH;

pole *wielkość segmentu* jest 20-bitowe, przy czym bit G = 1, więc obliczoną wartość należy przesunąć o 12 bitów w prawo — w rezultacie pole *wielkość segmentu* powinno zawierać liczbę FFFF8H;

- ♦ pozostałe pola deskryptora wypełnia się w konwencjonalny sposób; utworzony deskryptor opisuje segment rozszerzalny w dół, w którym rzeczywiste wartości adresu bazowego i rozmiaru są następujące:
 adres bazowy: 00017000H
 rozmiar: FFFF8FFFH
- ♦ z kolei inicjalizacja stosu wymaga wpisania do rejestru SS selektora, który wskazywać będzie utworzony deskryptor; do rejestru ESP należy wpisać 0;

♦ przykładowa realizacja instrukcji push eax

1. zmniejszenie ESP o 4, czyli

$$00000000H - 4 = FFFFFFFFCH$$

2. sprawdzenie czy odwołanie jest legalne, tzn. czy offset adresowanego obiektu jest **większy** od rozmiaru segmentu

$$FFFFFFFCH > FFFF8FFFH$$

3. obliczenie adresu liniowego w przestrzeni adresowej (adres bazowy + adres wirtualny):

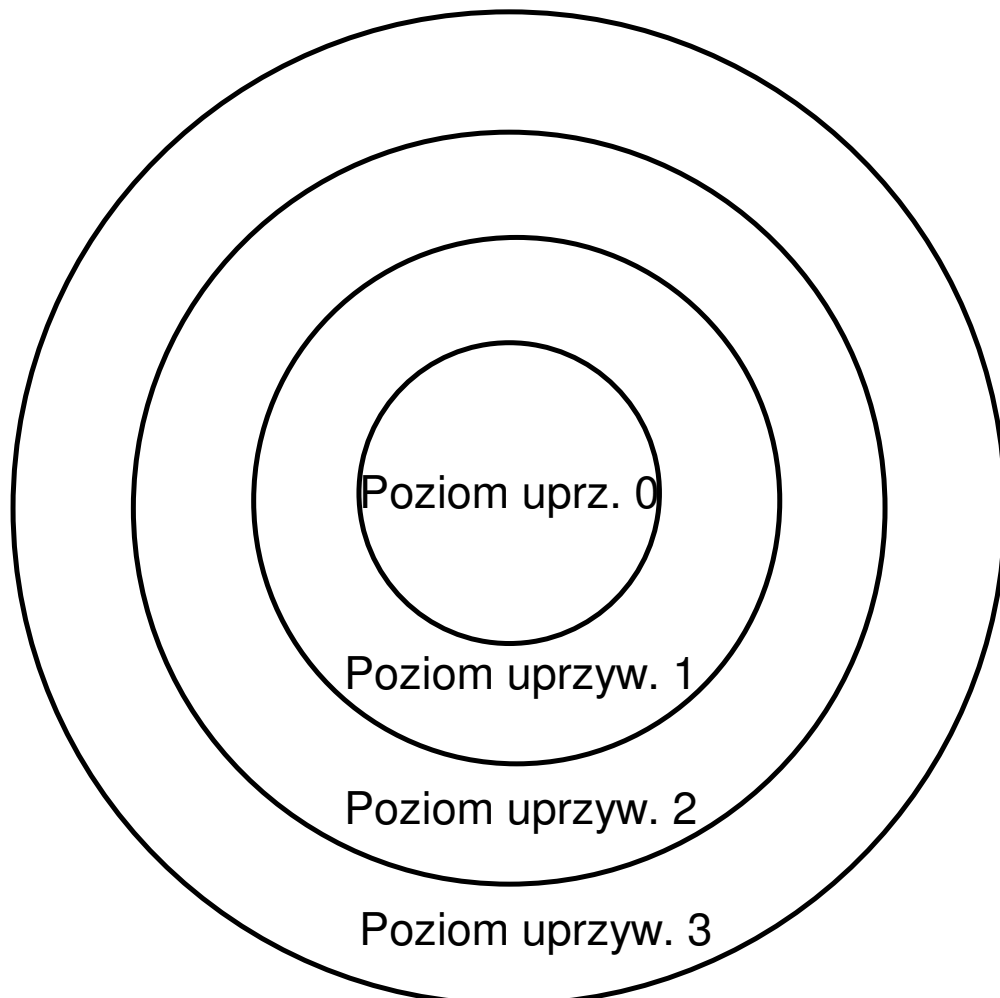
$$00017000H + FFFFFFFFCH = 100016FFCH$$

ponieważ najstarsza cyfra nie mieści się w zakresie 32-bitowym więc otrzymujemy adres liniowy = 00016FFCH;

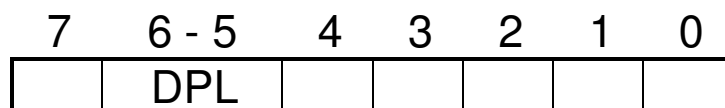
4. jeśli mechanizm stronicowania (omawiany dalej) nie jest aktywny, to uzyskany adres jest także adresem fizycznym wskazującym lokację pamięci, do której zostanie zapisana zawartość rejestru EAX.

Mechanizmy ochrony systemu operacyjnego i programów

- ◆ istotnym elementem pracy wielozadaniowej jest izolacja poszczególnych zadań i ochrona systemu operacyjnego; w szczególności wymaga to udostępnienia przez procesor środków technicznych, które pozwalałyby na nadanie systemowi operacyjnemu specjalnych przywilejów, niedostępnych dla zwykłych programów; zatem:
 - niektóre instrukcje (rozkazy) nie mogą być wykonywane przez zwykłe programy;
 - niektóre segmenty, dostępne dla systemu operacyjnego, powinny być niedostępne dla zwykłego programu;
 - wydzielone fragmenty systemu operacyjnego (np. podprogramy obsługi urządzeń) mogą być wywoływane przez zwykłe programy, ale tylko poprzez legalne punkty wejścia;



- ♦ w architekturze IA-32 zdefiniowano 4 poziomy uprzywilejowania, oznaczone numerami 0, 1, 2 i 3; najbardziej uprzywilejowany jest poziom nr 0 – na tym poziomie pracują podstawowe moduły systemu operacyjnego, zaś najmniej uprzywilejowany jest poziom 3, przeznaczony dla zwykłych programów; poziomy pośrednie przeznaczone dla oprogramowania narzędziowego, podsystemów zarządzania bazami danych. itp.;
- ♦ kody określające poziom uprzywilejowania występują w kilku strukturach systemowych:
 - każdy deskryptor segmentu zawiera bajt dostępu, w którym dwa bity, oznaczone symbolem DPL (ang. descriptor privilege level) określają "ważność" tego segmentu;



- każdy selektor zawiera pole RPL (ang. requestor's privilege level):



- aktualnie wykonywany kod programu ma przypisany poziom uprzywilejowania oznaczony jako CPL (ang. current privilege level); CPL jest określony przez pole DPL w deskrytorze segmentu, który jest wskazywany przez CS; w trakcie wykonywania programu procesor usuwa bity RPL w rejestrze CS i na ich miejsce wprowadza CPL;

♦ działanie mechanizmu ochrony określają cztery poniższe reguły:

1. załadowanie rejestru segmentowego wskazującego na segment danych jest możliwe tylko wówczas, gdy

$$DPL \geq CPL$$

gdzie:

DPL – poziom uprzywilejowania segmentu danych,

CPL – bieżący poziom uprzywilejowania;

(segmentom bardziej uprzywilejowanym odpowiadają kody o mniejszych wartościach); oznacza to, że dane mogą być zapisywane lub odczytywane tylko z takich segmentów, które nie są bardziej uprzywilejowane niż aktualnie wykonywany segment kodu;

2. w pewnych okolicznościach program może zmniejszyć swoje uprzywilejowanie zmieniając odpowiednio pole RPL w selektorze — w takim przypadku podana wyżej formuła przyjmuje postać

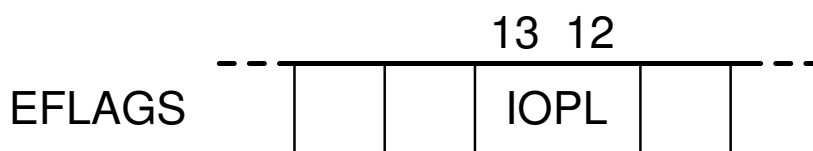
$$DPL \geq \max (CPL, RPL)$$

zagadnienie to będzie omawiane szerzej w dalszej części wykładu;

3. program wykonywany na pewnym poziomie uprzywilejowania nie może wywoływać bezpośrednio kodu bardziej uprzywilejowanego (za pomocą instrukcji CALL FAR lub JMP FAR), chyba że wywoływany kod znajduje się w segmencie zgodnym (ang. conform); kod na wyższym poziomie uprzywilejowania może być jednak legalnie wywoływany za pośrednictwem:
 - furtki wołania,
 - furtki przerwania,
 - furtki potrzasku,
 - furtki zadania;
4. program wykonywany na określonym poziomie uprzywilejowania nie może wywoływać kodu mniej uprzywilejowanego.

Przywileje poziomu 0

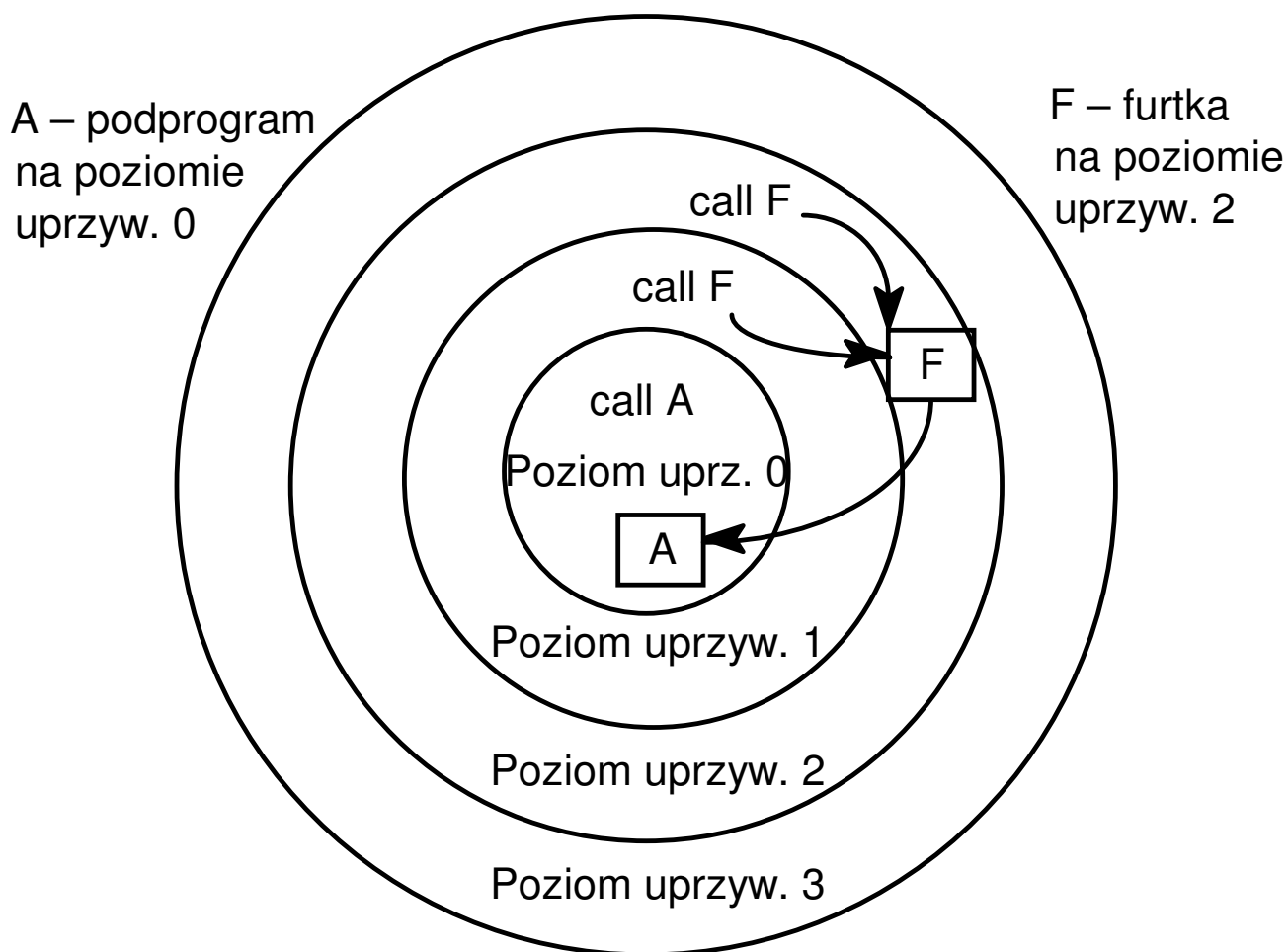
- ◆ wymienione instrukcje mogą być wykonywane tylko na poziomie 0: LIDT, LLDT, LGDT, LTR, LMSW, SMSW, CLTS, HLT oraz rozkazy realizujące przesłania do/z rejestrów CRn, DRn, TRn;
- ◆ dla rozkazów realizujących operacje w przestrzeni adresowej portów można wskazać poziom uprzywilejowania, począwszy od którego będą one dozwolone; poziom ten określony jest przez zawartość pola IOPL (bity 13-12) w rejestrze znaczników EFLAGS;



- ◆ pole IOPL określa także legalność rozkazów CLI i STI – wykonanie takiego rozkazu na poziomie mniej uprzywilejowanym niż określa IOPL powoduje wygenerowanie wyjątku;
- ◆ w odniesieniu do rozkazów IN, OUT, INS, OUTS dla każdego realizowanego zadania można określić tablicę bitową (ang. I/O Permission BitMap) specyfikującą porty, w odniesieniu do których podane rozkazy mogą być wykonywane – w ten sposób można selektywnie wskazać porty dozwolone i zabronione;
- ◆ rejestr znaczników można załadować za pomocą rozkazu POPF czy IRET – rozkazy te nie zmieniają jednak zawartości pola IOPL, jeśli wykonywane są na poziomie innym niż 0;
- ◆ podobne ograniczenia dotyczą znacznika zezwolenia na przerwanie IF: zmiana stanu IF na poziomie mniej uprzywilejowanym niż IOPL jest nieskuteczna.

Wołania podprogramów bardziej uprzywilejowanych

- ♦ wywoływanie podprogramów na wyższych poziomach uprzywilejowania możliwe jest tylko za pośrednictwem specjalnych deskryptorów zwanych *furtkami wołania* (ang. call gates) – furtki wołania, przechowywane w tablicy deskryptorów, stanowią jedyne legalne punkty wejścia do podprogramów;
- ♦ każda furtka ma przypisany pewien poziom uprzywilejowania; furtka wołania jest dostępna dla wszystkich programów uprzywilejowanych nie gorzej niż poziom furtki; wywołanie podprogramu za pośrednictwem furtki wołania polega na wykonaniu rozkazu **CALL** typu FAR: pole selektora tego rozkazu wskazuje furtkę, a offset jest ignorowany;



- ♦ dokładne położenie wywoływanego podprogramu podane jest wewnątrz furtki poprzez selektor do segmentu pamięci i offset wewnątrz tego segmentu;

bity 31 - 16 przesunięcia			+
bajt dostępu	0 0 0	liczba	+
selektor			+
bity 15 - 0 przesunięcia			+
			0

7	6 - 5	4	3	2	1	0
P	DPL	0	B	1	0	0

- ♦ pole DPL furtki określa konieczny stopień uprzywilejowania dla programu, który korzysta z furtki; zatem furtka może być użyta tylko wówczas, gdy $DPL \geq CPL$, gdzie CPL oznacza bieżący poziom uprzywilejowania;
- ♦ bit B w bajcie dostępu furtki przyjmuje wartość 0 dla furtki typu 286 i 1 dla 386 i wyższych.

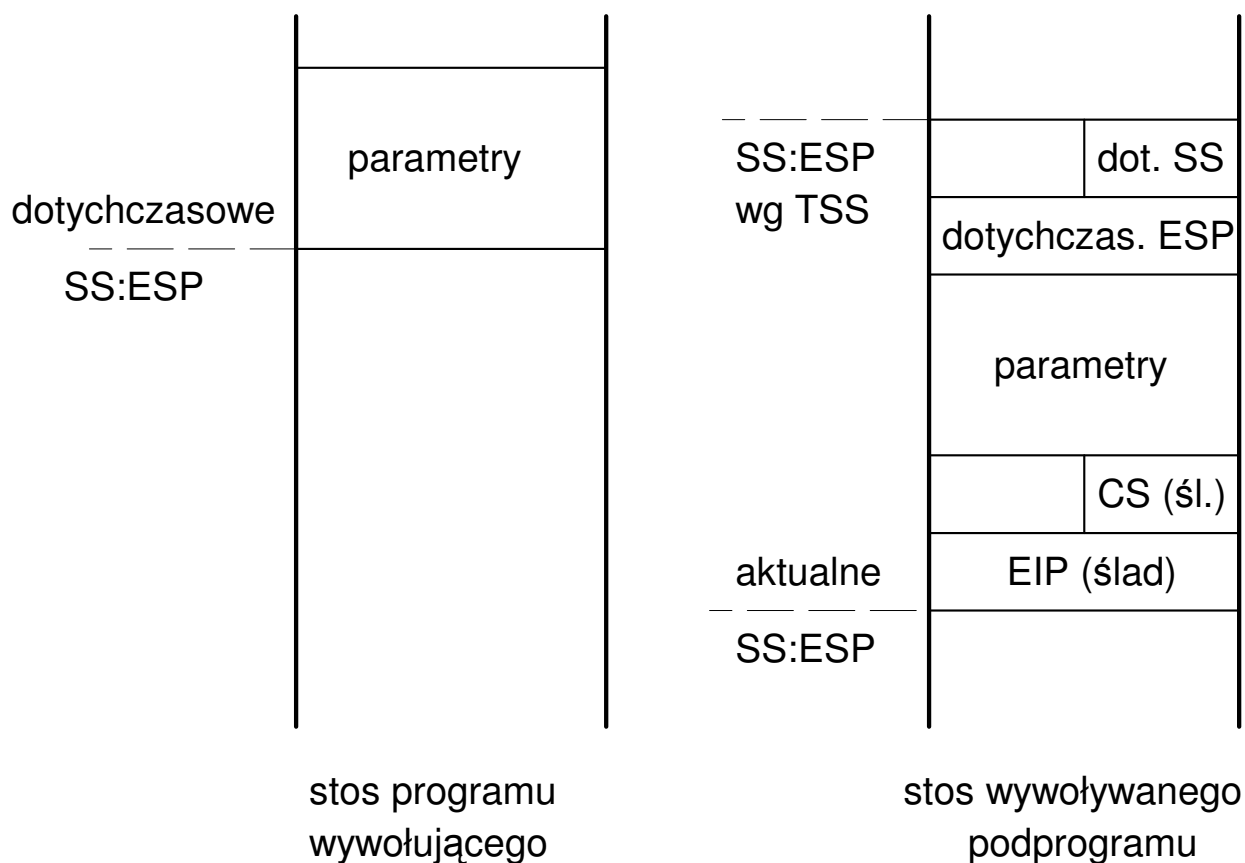
Wywoływanie i powrót z podprogramu

- ◆ w architekturze IA-32 wywołanie podprogramu w trybie chronionym wymaga rozpatrzenia trzech przypadków:
 1. program wywołujący i wywoływany podprogram są jednakowo uprzywilejowane;
 2. program wywołujący jest mniej uprzywilejowany niż wywoływany podprogram;
 3. program wywołujący jest bardziej uprzywilejowany niż wywoływany podprogram.
- ◆ jeśli program wywołujący i wywoływany podprogram są jednakowo uprzywilejowane (1), to wywołania i powroty realizowane są w sposób konwencjonalny; dotyczy to także przypadku, gdy wołanie przekazywane jest przez furtkę;
- ◆ jeśli program wywołujący jest mniej uprzywilejowany niż program wywoływany (2), to pośrednictwo furki wołania jest obowiązkowe; ponadto wywołany program powinien używać innego segmentu stosu, o rozmiarach dostosowanych do potrzeb programu (stos programu wywołującego jest czasami zbyt mały);
- ◆ jeśli (3) program wywołujący jest bardziej uprzywilejowany niż wywoływany podprogram (w szczególności może to oznaczać, że system operacyjny wywołuje mniej uprzywilejowany podprogram), to takie wywołania nie są dozwolone;
- ◆ w przypadku (3) użytkownik działający w złej intencji mógłby bowiem zmienić, wewnątrz mało uprzywilejowanego podprogramu, ślad używany przez instrukcję RET; wskutek tego sterowanie powróciłoby do bardziej uprzywilejowanego programu do miejsca stanowiącego nielegalny punkt wejścia – tym samym użytkownik mógłby uzyskać pewną kontrolę nad pracą systemu operacyjnego;

- ◆ z podobnych powodów zabronione są także skoki dalekie (JMP FAR) przez furtki do podprogramów o niższym poziomie uprzywilejowania (program wywołujący mógłby wówczas odpowiednio "spreparować" ślad, który prowadził by do zastrzeżonego kodu).

Przełączanie stosu przy wywołaniu podprogramów przez furtkę

- ◆ w przypadku wywoływania podprogramu za pośrednictwem furtki, gdy program wywołujący jest mniej uprzywilejowany niż wywołany podprogram, to wywołany podprogram powinien używać innego segmentu stosu; taka sytuacja występuje często także w przypadku wystąpienia przerwania lub wyjątku (zob. dalszy opis), ponieważ podprogramy obsługi przerw i wyjątków prawie zawsze wykonywane są na poziomie uprzywilejowania 0;
- ◆ nowe wartości rejestrów **SS** i **ESP** zawarte są w segmencie stanu zadania **TSS** (ang. task state segment); **TSS** zawiera 3 pary liczb określających zawartości **SS** i **ESP**, po jednej dla każdego poziomu uprzywilejowania 0, 1 i 2;
- ◆ po załadowaniu nowych wartości do **SS** i **ESP** (**SP**), procesor ładuje poprzednie zawartości **SS** i **SP** (**ESP**) na nowy stos; następnie ze starego do nowego stosu kopiowane są parametry podprogramu; pole licznikowe furtki określa ilość przesyłanych słów; w końcu na stos zapisywany jest ślad w postaci dotychczasowych zawartości **CS** i **EIP** (**IP**).



- ◆ wskutek zastosowania automatycznego kopiowania parametrów wywołana procedura może pracować w taki sam sposób, niezależnie od tego czy została wywołana z tego samego czy z innego poziomu (zauważmy, że obu tych przypadkach zawartość stosu jest jednakowa począwszy od 'parametrów').
- ◆ instrukcja **RET** wykonywana na końcu podprogramu musi posiadać bliższe informacje o zastosowanym sposobie wywołania – informacje te można uzyskać poprzez porównanie aktualnego poziomu uprzywilejowania CPL z wartością CPL programu wywołującego (wartość CPL programu wywołującego została zapisana na stosie w polu śladu (rejestr CS)).

Kontrola poprawności wskaźników

- ◆ przypuśćmy, że w systemie operacyjnym, na poziomie uprzywilejowania 1, dostępny jest podprogram DATA_CZAS, który wpisuje aktualną datę i czas do wskazanego obszaru pamięci (obszar ten wskazywany jest przez parametr podprogramu postaci 'selektor : przesunięcie');
- ◆ przypuśćmy dalej, że program na poziomie 3, działający w złej intencji, wywołuje poprzez furtkę podprogram DATA_CZAS, podając jako parametr wskaźnik do obszaru w segmencie o poziomie uprzywilejowania 2 (obszar ten jest niedostępny dla programu na poziomie 3, natomiast dostępny jest dla podprogramu DATA_CZAS, który wykonywany jest na poziomie 1) – zatem, na zlecenie programu z poziomu 3 zostanie wykonany niedozwolony zapis do segmentu na poziomie 2;
- ◆ ochrona przed takim atakiem wymaga wprowadzenia do podprogramu odpowiednich mechanizmów kontrolnych — wyniki podprogramu mogą być zapisywane tylko do segmentów danych uprzywilejowanych tak samo lub mniej niż program wywołujący;
- ◆ warunkiem dostępu do segmentu jest aby

$$DPL \geq \max (CPL, RPL)$$

gdzie RPL jest polem w selektorze przekazanym jako parametr do podprogramu;

- ◆ jeśli wywołany podprogram wpisze do pola RPL poziom uprzywilejowania programu wywołującego (służy do tego instrukcja ARPL), to efektywny poziom uprzywilejowania, będzie zdeterminowany przez program mniej uprzywilejowany.

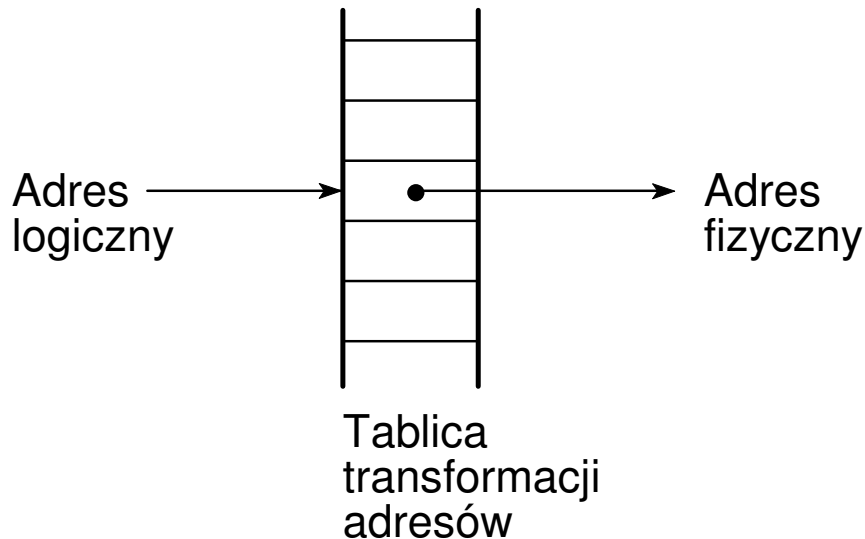
Segmenty zgodne

- ◆ istnieje duża grupa podprogramów systemowych, które nie wymagają specjalnych przywilejów i mogą być wykonywane na poziomie programu wywołującego;
- ◆ takie podprogramy można umieścić w specjalnie do tego celu zaprojektowanych segmentach określanych jako segmenty zgodne (ang. conforming); segmenty tego typu są identyfikowane przez bit $C = 1$ w bajcie dostępu deskryptora segmentu kodu;
- ◆ w segmentach zgodnych obowiązują odmienne reguły wywoływania podprogramów — mogą one być wywoływane bezpośrednio z dowolnego programu, bez pośrednictwa furtek, jednak pod warunkiem, że wywoływane podprogramy znajdują się w segmencie tak samo lub bardziej uprzywilejowanym niż program wywołujący (tj. DPL (dla segmentu zgodnego) $\leq$ CPL (dla programu wywołującego)).

Stronicowanie

- ◆ w trakcie wieloletniego rozwoju systemów komputerowych zaobserwowano szereg rozmaitych trudności związanych z racjonalnym zarządzaniem pamięcią, zwłaszcza w systemach wielozadaniowych;
- ◆ omawiany wcześniej mechanizm segmentacji realizuje odwzorowanie adresów poprzez sumowanie adresu wirtualnego z adresem bazowym podanym w deskryptorze segmentu — uzyskany w ten sposób adres liniowy, o ile stronicowanie jest wyłączone, jest adresem fizycznym;
- ◆ jednak takie odwzorowanie adresów jest dość prymitywne i nie rozwiązuje szeregu problemów związanych z zarządzaniem pamięcią; istotne trudności spowodowane są niejednakowymi rozmiarami segmentów — ich wymiana między pamięcią operacyjną i dyskową powoduje powstawanie znacznej liczby rozproszonych obszarów pamięci, które nie znajdują zastosowania ze względu na zbyt małe rozmiary;
- ◆ analiza tych problemów wskazuje iż najlepszym rozwiązaniem byłoby skonstruowanie mechanizmu, który pozwalał by tłumaczyć adres logiczny (wirtualny) na dowolny adres fizyczny; wówczas pamięć widziana przez program stanowiła by zwarty obszar pamięci, podczas gdy rzeczywiste lokacje pamięci tworzące ten obszar byłyby rozproszone w całej pamięci fizycznej;
- ◆ zwiększenie obszaru danych programu byłoby możliwe, jeśli tylko w pamięci operacyjnej dostępne byłyby aktualnie nieprzydzielone lokacje o dowolnych adresach; co więcej za pomocą takiego mechanizmu można by wykonywać nawet programy odwołujące się do lokacji pamięci o adresach nie istniejących w zainstalowanej pamięci RAM — odwołania te musiały być skierowane do odpowiednich, rzeczywistych komórek pamięci;

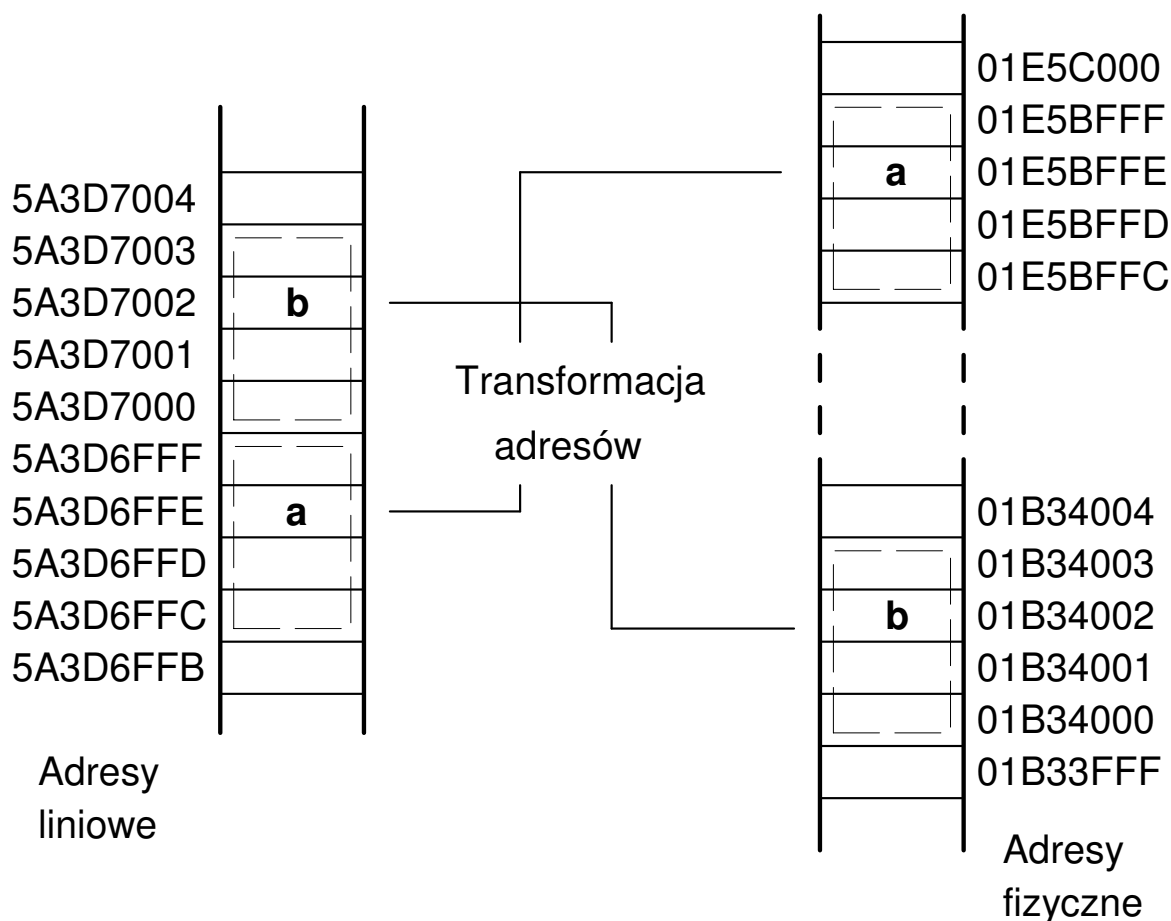
- ♦ transformacja taka mogłaby być realizowana za pomocą tablicy transformacji adresów, w której adres logiczny stanowił by indeks wskazujący element tablicy, w którym zapisany jest wymagany adres fizyczny;



- ♦ w przypadku 4-gigabajtowej przestrzeni adresowej, pokazana tablica musiałaby zawierać 2^{32} elementów, przy czym każdy element zawierał by 4 bajty; pomijając rozmiar takiej tablicy należy także brać pod uwagę konieczność ciągłej aktualizacji tablicy w zależności od wymagań stawianych przez poszczególne zadania;
- ♦ z podanych przyczyn transformacja adresów prowadzona jest w sposób nieco uproszczony; zazwyczaj transformowany adres logiczny dzieli się na dwa lub trzy pola; pole zawierające kilkanaście najmniej znaczących bitów adresu nie jest transformowane, natomiast pozostałe pola są przekształcane za pomocą tablicy;
- ♦ w takim ujęciu adres uzyskany po transformacji starszych bitów adresu pierwotnego wskazuje pewien obszar pamięci fizycznej, nazywany *stroną*, w którym położenie konkretnej lokacji pamięci określa młodsza, nietransformowana część adresu pierwotnego;

- ♦ rozmiar strony zależy od liczby bitów, która nie podlega transformacji; w architekturze IA-32 najczęściej stosuje się schemat transformacji, w którym 12 najmniej znaczących bitów adresu liniowego nie jest przekształcane — zatem w tym przypadku strony mają rozmiar 4096 bajtów (czyli 4 KB); dostępne są też schematy transformacji, w których strony mają rozmiar 2 lub 4 MB;
- ♦ w takim ujęciu, w przypadku stron 4 KB, tablica transformacji odwzorowuje 4-kilobajtowe bloki liniowej przestrzeni adresowej w 4-kilobajtowe bloki fizycznej przestrzeni adresowej; zatem w tablicy transformacji każda pozycja związana jest z jedną stroną 4 KB.

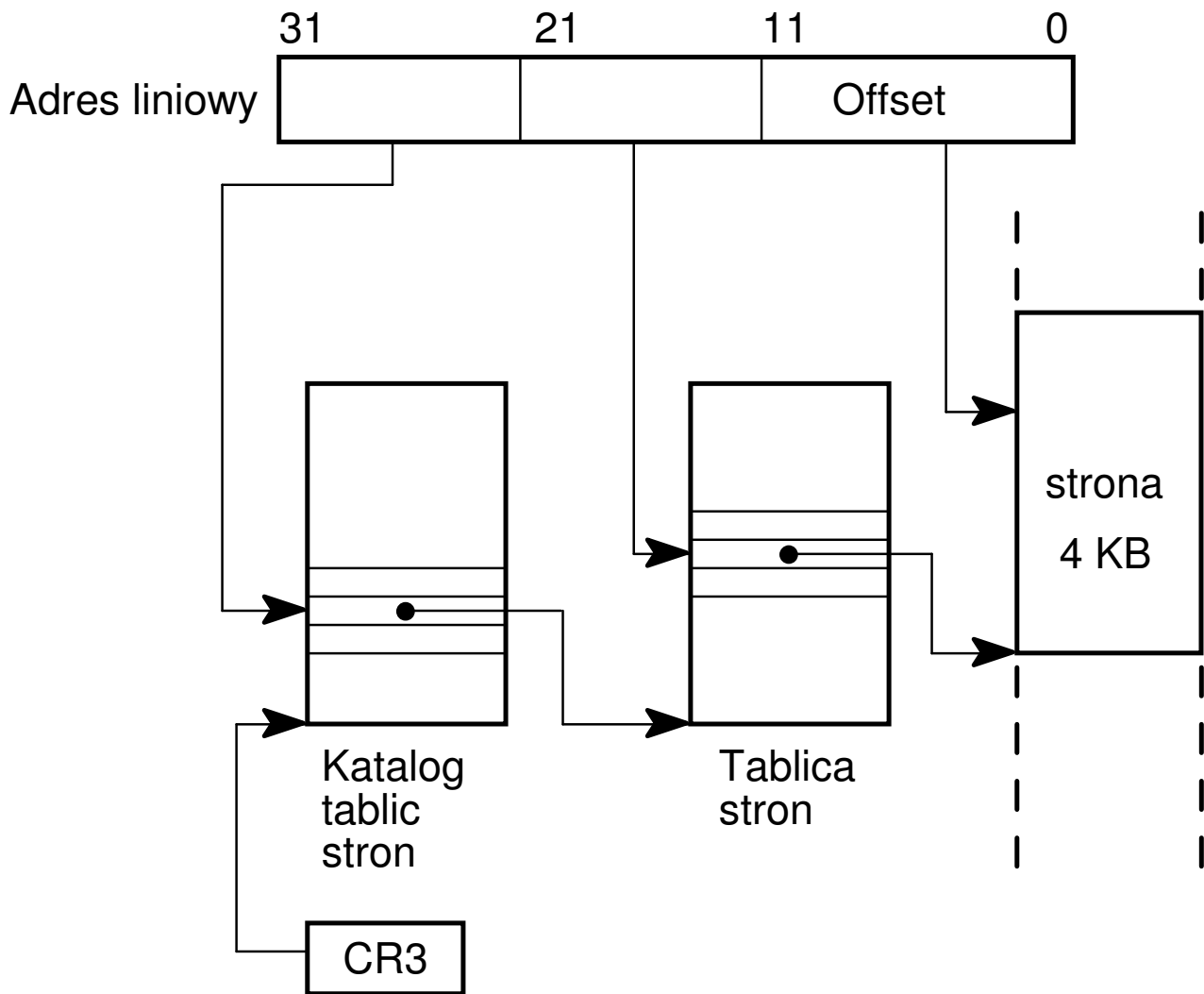
Przykład — z punktu widzenia programu zmienne **a** i **b** zajmują sąsiednie lokacje pamięci, chociaż w rzeczywistości (w pamięci fizycznej) zajmują odległe lokacje.



- ◆ wybór wielkości strony determinowany jest przez niżej wymienione czynniki:
 - pożądaną jest by strony miały jednakową długość – ułatwia to projektowanie sprzętu, zapobiega powstawaniu małych i trudnych do zagospodarowania obszarów, a algorytmy obsługi pamięci o jednakowych stronach są łatwiejsze do realizacji;
 - ze względu na konieczność wymiany stron między pamięcią dyskową i operacyjną rozmiar strony powinien być wielokrotnością rozmiaru sektora dyskowego;
 - jeśli strona jest zbyt duża, to mogą występować zbędne transmisje dużych fragmentów pamięci, jeśli zaś strona jest zbyt mała, to liczba transmisji może być nadmierna;
 - w architekturze IA-32 przyjęto standardowe rozmiary stron: 4 kB, 2 MB i 4 MB; w starszych typach procesorów stosowano wyłącznie strony 4 kB.

Dwupoziomowe tablice stron

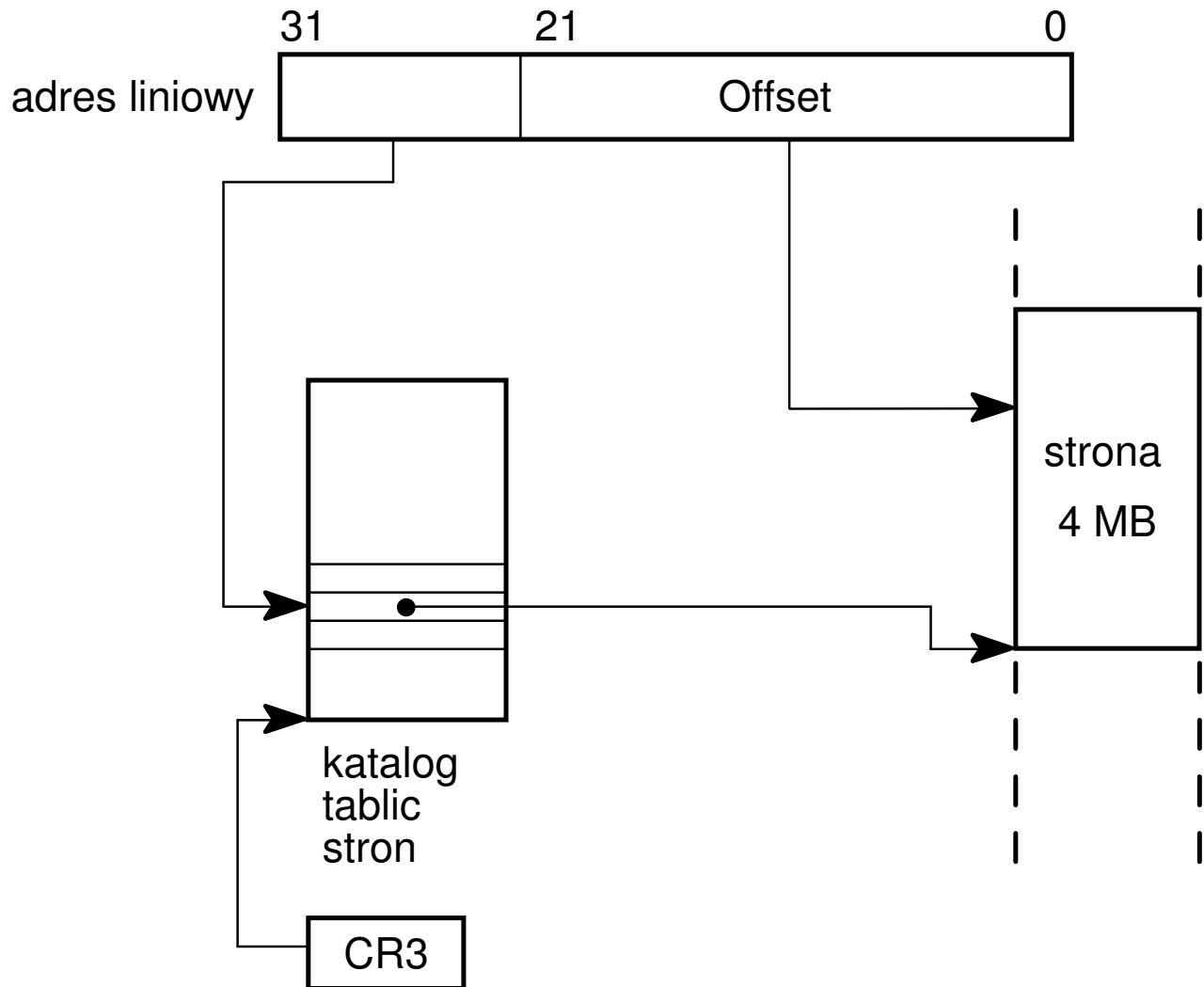
- ◆ w przestrzeni adresowej 4 GB (liniowej lub fizycznej) można zdefiniować 2^{20} stron po 4 KB – zatem tablica transformacji adresów powinna zawierać 2^{20} pozycji, przy czym każda pozycja powinna zawierać 20-bitowy adres strony (strony zaczynają się od adresów podzielnych przez 4096, więc 12 najmłodszych bitów zawiera zawsze zera);
- ◆ obok numeru strony trzeba też przechowywać inne informacje, więc na opis pojedynczej pozycji przeznaczono 32 bity (4 bajty);
- ◆ tak zorganizowana tablica translacji zajmowałaby $2^{20} * 4$ bajtów, czyli 4 MB; w połowie lat dziewięćdziesiątych w większości komputerów PC rozmiar zainstalowanej pamięci operacyjnej wynosił 4 MB; z tego powodu dla stron 4 KB wprowadzono dwupoziomą organizację tablic transformacji:
 - poziom wyższy tworzy *katalog tablic stron* — jest to tablica, która dzieli 4 GB liniową przestrzeń adresową na 1024 grupy stron; zatem tablica ma 1024 pozycje 32-bitowe; położenie katalogu tablic stron określa rejestr CR3;
 - poziom niższy tworzą *tablice stron* — mają format identyczny z katalogiem stron (1024 pozycje po 32 bity); tablica stron dzieli 4 MB grupę stron na 1024 strony po 4 kB;
- ◆ wprowadzenie dwupoziomowej organizacji tablic stron implikuje podział 32-bitowego adresu liniowego na trzy elementy:
 - 10-bitowe pole wskazujące pozycję katalogu tablic stron,
 - 10-bitowe pole wskazujące pozycję w tablicy stron,
 - 12-bitowe pole, które nie podlega żadnym przekształceniom i wskazuje offset względem początku strony;ilustruje to poniższy rysunek:



- ♦ rejestr CR3 zawiera adres fizyczny katalogu tablic stron (ściśle: adres podany na bitach $31 \div 12$ jest rozwijany do adresu 32-bitowego poprzez dołączenie 12 najmłodszych bitów o wartości 0);
- ♦ mechanizm stronicowania korzysta także z rejestru CR2 — rejestr ten zawiera 32-bitowy adres liniowy, który spowodował wystąpienie błędu stronicowania (zob. dalszy opis);
- ♦ stronicowanie w architekturze IA-32 dostępne jest tylko w trybie chronionym i stanowi mechanizm opcjonalny, który może być aktywny lub nie (w zależności od stanu bitu PG (ang. paging) w rejestrze CR0);

- ◆ mechanizm stronicowania jest niezależny od segmentacji; w przypadku segmentacji obszary pamięci przesyłane do/ dysku mają zmienny rozmiar (wielkość segmentu); w przypadku stronicowania rozmiar przesyłanego obszaru jest stały;
- ◆ w systemie Windows każdy proces ma przyporządkowany własny katalog tablic stron; zatem w chwili przełączania procesów zmienia się również zawartość rejestru CR3.

Jednopoziomowe tablice stron (strony 4 MB)



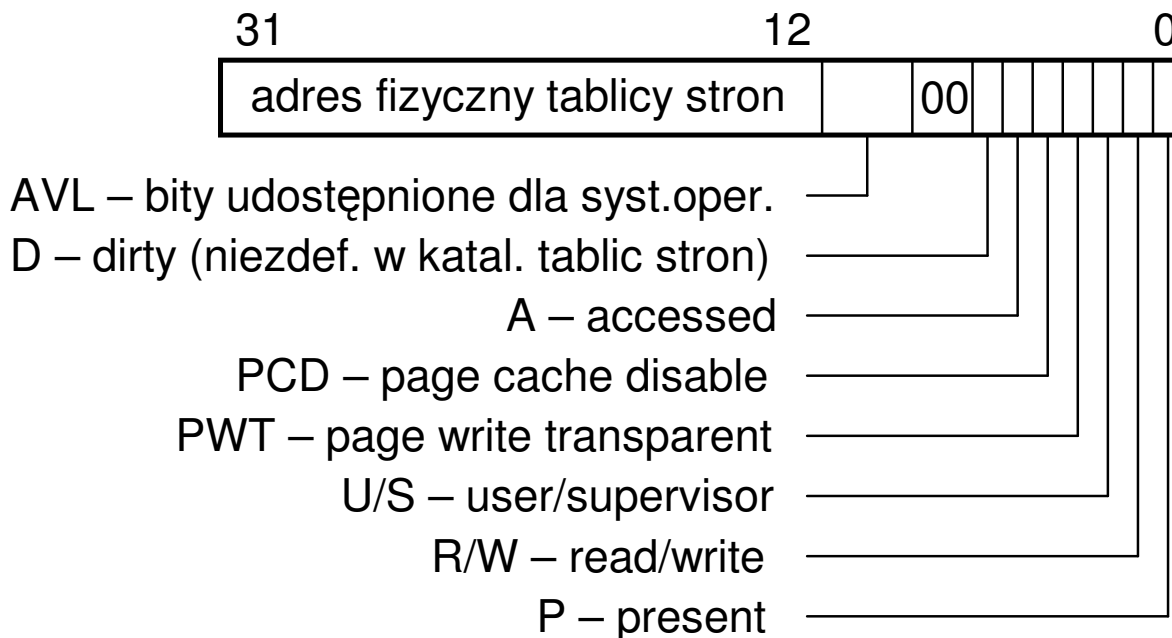
Realizacja pamięci wirtualnej za pomocą stronicowania

- ◆ stronicowanie pozwala na symulowanie dużej pamięci operacyjnej za pomocą stosunkowo niedużej pamięci RAM i pamięci dyskowej; mechanizm ten polega na przechowywaniu zawartości pamięci symulowanej częściowo w pamięci RAM i częściowo w pamięci dyskowej;
- ◆ jeśli w trakcie wykonywania programu potrzebna jest strona, która w danej chwili nie znajduje się w pamięci RAM, to inna, aktualnie nieużywana strona kopiowana jest na dysk, a na jej miejsce wprowadzana jest żądana strona; w systemie Windows strony zapisywane i odczytywane z dysku gromadzone są w *pliku wymiany*;
- ◆ w architekturze IA-32 zdefiniowano 4-gigabajtową przestrzeń adresową — pola różnych typów deskryptorów są przystosowane do przechowywania adresów 32-bitowych; w szczególności w trakcie wykonywania instrukcji tworzony jest 32-bitowy *adres liniowy* — adres ten poprzez mechanizm stronicowania powinien zostać przetworzony na adres fizyczny, wskazujący rzeczywistą komórkę pamięci; jeśli strona, do której następuje odwołanie, aktualnie nie znajduje się w pamięci operacyjnej, to generowany jest wyjątek (przerwanie), który obsługiwany jest przez system operacyjny;
- ◆ realizacja pamięci wirtualnej za pomocą stronicowania ma liczne zalety, wśród których można wymienić:
 - poszczególne procesy mają dostęp wyłącznie do swoich własnych obszarów pamięci — jakiegokolwiek odwołanie do pamięci przechodzi zawsze przez tablice transformacji adresów i jest kierowane do własnego obszaru pamięci procesu;
 - zarządzanie pamięcią jest znacznie usprawnione, ponieważ system operacyjny nie musi przeprowadzać scalania małych, niezajętych obszarów pamięci; zwarty obszar pamięci

wirtualnej może być ulokowany w postaci wielu stron rozproszonych w całej pamięci RAM.

Katalog tablic stron

- ◆ każda pozycja w katalogu tablic stron zawiera:
 - w przypadku stron 4 KB — położenie tablicy stron,
 - w przypadku stron 4 MB — położenie pojedynczej strony 4 MB;



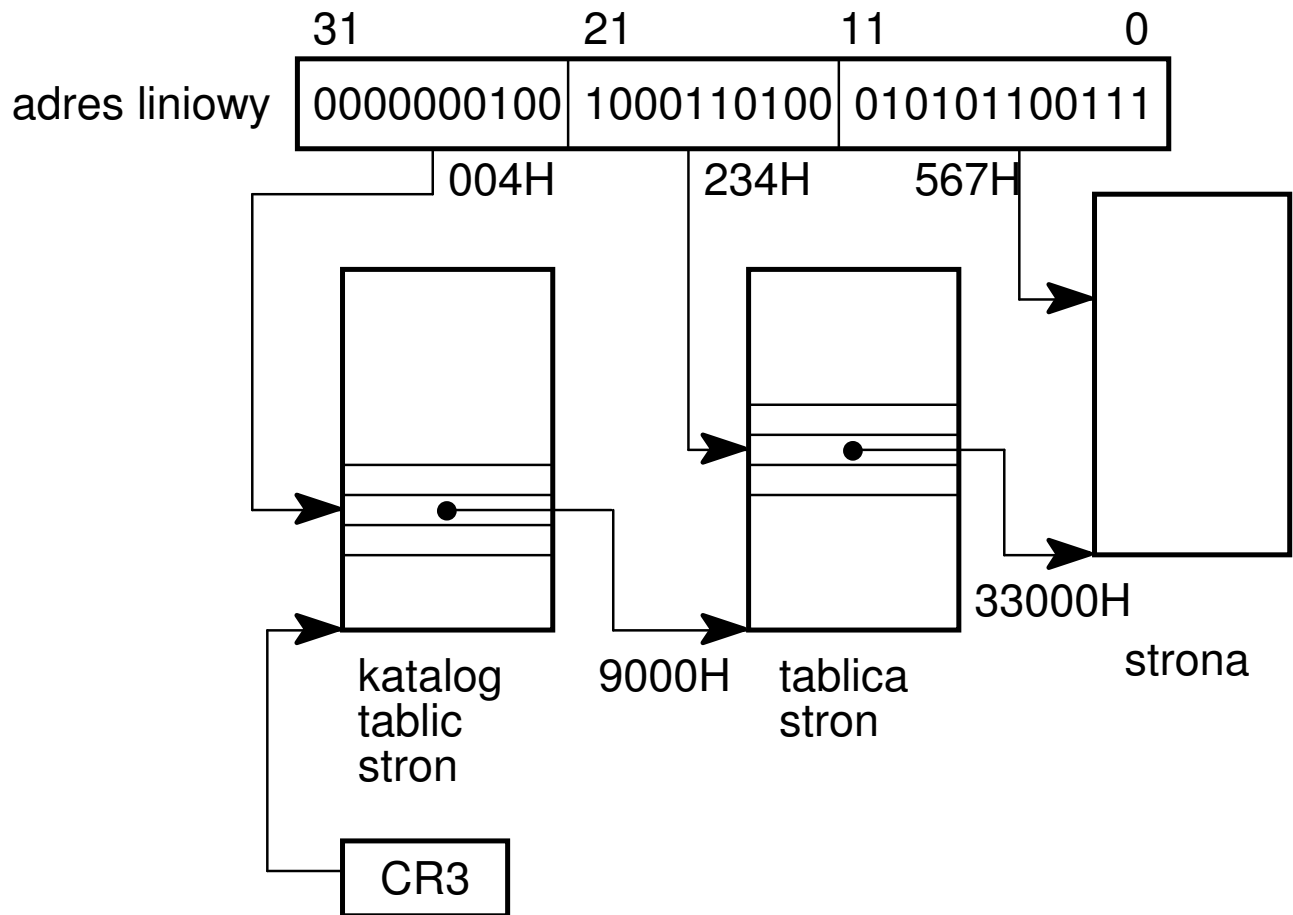
- ◆ pole "adres fizyczny tablicy stron" określa fizyczne położenie tablicy stron dla tej grupy stron – pole to rozszerzane jest do 32 bitów poprzez dołączenie 12 bitów zerowych z prawej strony (tablice stron muszą być lokowane począwszy od adresów będących wielokrotnościami 4 kB);
- ◆ bity:
 - PCD — blokady zapisu strony do pamięci podręcznej,
 - PWT — jednoczesnego zapisywania strony w pamięci głównej i podręcznej
 używane są w operacjach związanych z przechowywaniem stron w pamięci podręcznej (ang. cache memory); opis tych bitów podany jest dalej;

- ◆ bit A jest ustawiany w stan 1, gdy grupa stron została wybrana — na tej podstawie system operacyjny może gromadzić informacje dotyczące wykorzystania stron;
- ◆ bit U/S = 0 – grupa stron nie jest dostępna dla programów użytkowych, = 1 – dostępna;
- ◆ bit R/W = 0 – dozwolony tylko odczyt, = 1 – dozwolony odczyt i zapis;
- ◆ bit P wskazuje czy tablica stron dla tej grupy znajduje się aktualnie w pamięci operacyjnej (P = 1) czy też na dysku (P = 0).

Tablice stron

- ◆ tablice stron mają format identyczny z katalogiem stron (1024 pozycje po 32 bity); tablica stron dzieli 4MB grupę stron na 1024 strony po 4kB;
- ◆ bit A jest ustawiany (sprzętowo) w stan 1, gdy wykonywany jest dostęp do pamięci na danej stronie; zerowanie bitu A realizowane jest wyłącznie w sposób programowy; system operacyjny może wykorzystać ten bit do rejestrowania informacji o wykorzystaniu poszczególnych stron;
- ◆ bit D jest zerowany w chwili przepisania strony z dysku do pamięci operacyjnej; jeśli na danej stronie wykonywany jest zapis, to bit D ustawiany jest w stan 1; jeśli bit ten zawiera 0, to oznacza że zawartość strony w pamięci operacyjnej i zawartość na dysku są identyczne — jeśli zachodzi konieczność usunięcia strony, to kopiowanie na dysk jest zbędne;
- ◆ R/W = 0 – tylko odczyt, = 1 – odczyt i zapis; w systemie Windows w aplikacjach DOSowych (tzw. DOS-box) strony odwzorowane w pamięć obrazu mają R/W = 0, co pozwala na przechwycenie wszystkich zapisów do pamięci obrazu i przesłanie ich do odpowiednich okien zadań DOSowych (pracujących w trybie graficznym) — identyczny mechanizm omawiano wcześniej przy segmentacji.

Przykład



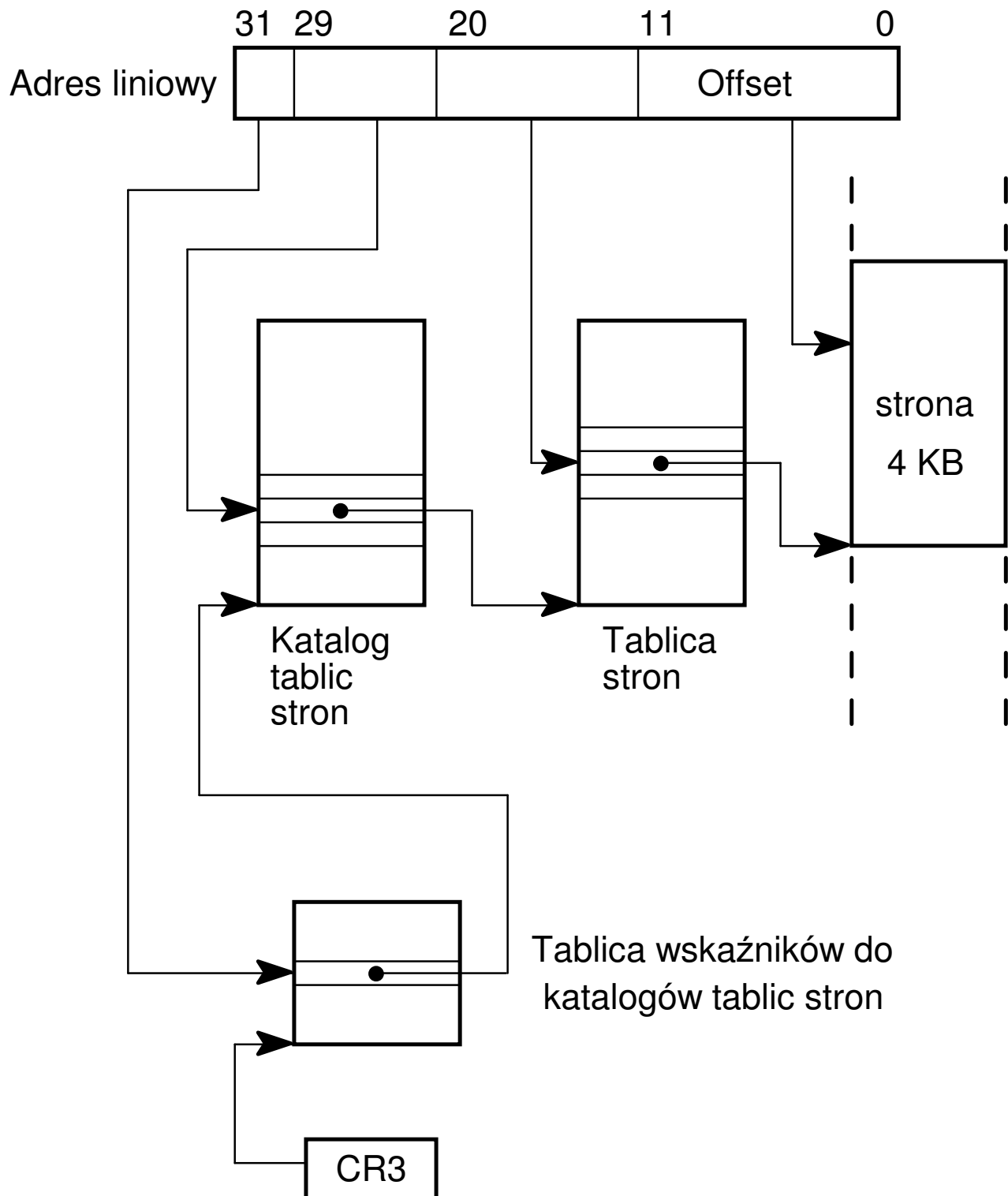
- ♦ wyznaczony adres fizyczny jest równy 00033576H.

Współpraca z pamięcią podręczną

- ◆ bit PCD (ang. page cache disable) w katalogu tablic stron pozwala na włączenie blokady zapisu strony do pamięci podręcznej, co może być konieczne w przypadku, np. odwzorowania rejestrów urządzeń w przestrzeni adresowej pamięci; ustawienie tego bitu powoduje wyłączenie umieszczania w pamięci podręcznej odpowiednich obszarów pamięci głównej;
- ◆ bit PWT (ang. page write transparent) określa sposób jednoczesnego zapisywania strony w pamięci głównej i w pamięci podręcznej; ustawienie tego bitu powoduje, że wszelkie zapisy do danej strony następują bezpośrednio do pamięci głównej, z pominięciem protokołu zapisu opóźnionego (ang. write-back); taki sposób konieczny jest, gdy pamięć jest współdzielona między procesorem a innymi układami komputera, które mogą odczytywać tę pamięć (np. karty graficzne);
- ◆ procesor używa bitów PCD i PWT tylko przy włączonej wewnętrznej pamięci podręcznej (bit CD w rejestrze CR0 równy 0); w przeciwnym przypadku procesor przyjmuje $PCD = 1$ i $PWT = 1$ powodując zablokowanie zewnętrznej pamięci podręcznej (sygnały PCD i PWT wyprowadzone są na zewnątrz układu).

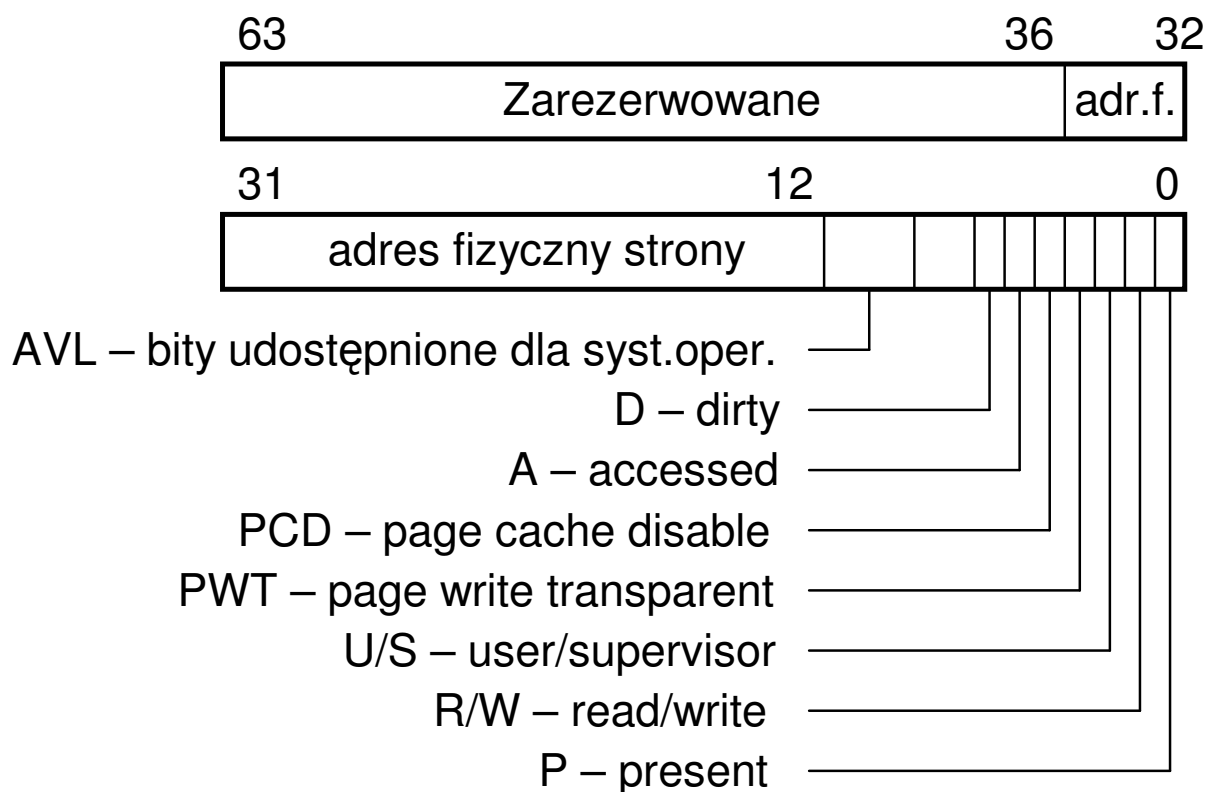
Stronicowanie w fizycznej przestrzeni adresowej 64 GB

- ◆ możliwość adresowania pamięci fizycznej o rozmiarze 64 GB została wprowadzona począwszy od procesora Pentium Pro; użycie opisanego dalej trybu adresowania, przy włączonym stronicowaniu (PG — bit nr 31 w rejestrze CR0), wymaga dodatkowo ustawienia bitu PAE (bit nr 5) w rejestrze CR4; możliwe jest tworzenie stron 4 KB lub 2 MB;
- ◆ w omawianym trybie pozycje w tablicach stron rozszerzono do 64 bitów; katalogi tablic stron i tablice stron zajmują nadal po 4 KB, ale mają tylko 512 pozycji;
- ◆ wprowadzono nową tablicę *wskazników do katalogów tablic stron* (ang. page directory pointer table); tablica ta ma 4 pozycje 64-bitowe;
- ◆ położenie nowej tablicy jest nadal wskazywane przez zawartość rejestru CR3, przy czym zwiększono do 27 liczbę interpretowanych bitów tego rejestru, co oznacza, że tablica może zaczynać się od adresu podzielonego przez 32;



- ◆ elementy podanych tablic są 64-bitowe; struktura elementu w tablicy stron podana jest na rysunku (elementy dwóch pozostałych tablic

mają zbliżoną strukturę); adres fizyczny strony zajmuje jest umieszczony na bitach 35 ÷ 12;



- ◆ możliwe jest również definiowanie stron 2 MB — wówczas bity 20 ÷ 0 adresu liniowego wskazują offset na stronie 2-megabajtowej;
- ◆ podany sposób wyznaczania adresu fizycznego oznacza, że przy ustalonej zawartości ww. tablic, możliwy jest dostęp do 4 GB pamięci fizycznej; dostęp do innych obszarów pamięci fizycznej (64 GB) wymaga włączenia innego zestawu tablic, co realizuje się poprzez zmianę zawartości rejestru CR3 lub poprzez zmianę wskaźników do katalogów tablic stron.

- ◆ jeśli opis pewnej strony znajduje się w TLB, to procesor pobiera informacje o adresie fizycznym wprost z TLB; przeglądanie tablicy TLB wykonują układy równoległe, które pozwalają na natychmiastowe stwierdzenie, czy potrzebna pozycja jest w TLB;
- ◆ stosowane są odrębne tablice TLB dla danych i dla rozkazów, jak również dla stron 4 KB i 4 MB (rozmiar tablicy można wyznaczyć za pomocą instrukcji CPUID);
- ◆ omawianą tablicę można traktować jako rodzaj pamięci adresowanej przez zawartość (tj. przez podanie adresu liniowego), czyli jest to *pamięć asocjacyjna (skojarzeniowa)*;
- ◆ tablica TLB nie jest automatycznie aktualizowana przy zmianie katalogu tablic stron lub tablicy stron — z tego powodu konieczne jest, w przypadku zmiany zawartości którejś z podanych tablic, usunięcie odpowiedniego wiersza z tablicy TLB;
- ◆ każdy zapis do rejestru CR3 powoduje usunięcie wszystkich elementów z tablicy TLB – często do rejestru CR3 wpisuje się tę samą wartość wyłącznie w celu "wyczyszczenia" TLB;
- ◆ dostępna jest instrukcja INVLPG, która usuwa pojedyncze pozycje w tablicy TLB;
- ◆ dodatkowo, dostępny jest mechanizm blokujący usuwanie wybranych wierszy TLB w przypadku zmiany zawartości rejestru CR3; blokada ta dotyczy zazwyczaj często używanych stron;
- ◆ jeśli przyjmiemy, że tablica TLB zawiera 32 wiersze, to tym samym wskazane są 32 strony, każda po 4 KB, a więc dostęp do najczęściej używanych stron o łącznym rozmiarze 128 KB nie wymaga wykonywania pełnej transformacji adresu.